DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

Hybrid-Sentinel: A Multi-Tiered AI Framework for NDN Intrusion Detection

A M.Tech Thesis

Submitted by

GUGULOTH RAJU (CS24M036)

For the award of the degree

Of

MASTER OF TECHNOLOGY

May 2026

THESIS CERTIFICATE

This is to undertake that the M.Tech Thesis titled **HYBRID-SENTINEL: A MULTI-TIERED AI FRAMEWORK FOR NDN INTRUSION DETECTION**, submitted by me to the Indian Institute of Technology Madras, for the award of **Master of Technology**, is a bonafide record of the research work done by me under the supervision of **Prof. Chester Rebeiro**. The contents of this M.Tech Thesis, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Chennai 600036 **GUGULOTH RAJU (CS24M036)**

Date: May 2026

Prof. Chester Rebeiro
Project Guide and Professor
Department of Computer Science and Engineering
IIT Madras

ACKNOWLEDGEMENTS

First of all, I want to express my heartfelt thanks to Prof. **Chester Rebeiro**, my academic guide at IIT Madras, for his unceasing guidance, great feedback, and support all the time during this project.

I appreciate all the support from my industry mentor, **Mr. Tushar Sood** from **Tata Communications**, located at IITM Research Park, who provided me with practical insights and was always there to support me. Moreover, I would like to thank **Tata Communications** for their guidance and for allowing me to work on this important problem.

I appreciate the support from **Tata Communications** that included data access, domain expertise, and resources that made this work possible. I am also thankful to IIT Madras and the **Department of Computer Science and Engineering** for granting me the use of their facilities and for creating an environment that is academically enriching.

Last but not least, I want to say thank you to my coworkers, friends, and family for being there for me and cheering me on during the entire process.

Raju Guguloth

ABSTRACT

Named Data Networking (NDN) replaces IP addressing with content names, and that single change closes off most of the inspection surface that legacy firewalls were built around. Three classes of attack become difficult to handle as a result. Interest Flooding exhausts the Pending Interest Table by sending request rates that stay just under per-source rate limits, so each request looks individually reasonable. Cache Pollution does the same to the Content Store by coordinating consumers whose behaviour is plausible only in isolation. Collusive variants of both attacks are graph-structural rather than packet-local, which is why per-flow inspection misses them. Signature engines have no string to match. Address-based filters have no addresses to match. Per-packet deep inspection is too slow for 100 Gbps forwarding.

This thesis presents **Hybrid-Sentinel**, a three-tier cascaded detection pipeline that addresses these gaps without sacrificing line-rate throughput. Tier-1 is a Random Forest classifier operating on a stateless 17-dimensional feature vector with IP addresses deliberately excluded; it resolves the majority of unambiguous traffic in under 5 ms. Tier-2 is a CNN-GRU that consumes temporally ordered 20-packet windows and learns the inter-arrival and entropy signatures of low-rate exhaustion attacks. Tier-3 is an Edge-Conditioned Bipartite Graph Autoencoder (EC-GAE). The EC-GAE builds a Consumer–Producer bipartite graph whose edge attributes are the 128-dimensional terminal GRU hidden states from Tier-2, trains on benign traffic only, and flags any flow whose reconstruction error exceeds the 95th-percentile benign threshold as a zero-day anomaly.

The Round 17 evaluation, run on the `v5_final` corpus collected from a Docker-emulated NDN-over-IP testbed and split with `GroupShuffleSplit` on the `(dst_port, ip_proto)` key to prevent leakage, reports a Macro F_1 of 0.984 across five attack classes. Slow-HTTP, Port-Scan, and DNS-Tunnelling reach perfect per-class precision, recall, and F_1. Aggregate per-packet inference latency is 4.57 ms, against 12.57 ms for the Mamba state-space-model baseline at comparable accuracy. The transition from Cross-Entropy to Focal Loss with $\gamma = 2$ between Round 13 and Round 16 alone accounts for a 0.30 gain in Macro F_1. The Tier-3 EC-GAE, trained on 4,200 benign flow sequences with no attack labels and evaluated on a held-out graph of 1,800 benign and 4,000 attack flows, reports a true-positive rate of 63.22%, a false-positive rate of 18.11%, and an ROC AUC of 0.8091. Because Tier-3 only sees the residual ambiguous traffic that Tier-1 and Tier-2 escalate (roughly 5% of the total stream), the 18.11% per-stage false-alarm rate corresponds to a system-level false-alarm rate near 1%.

KEYWORDS Named Data Networking; Intrusion Detection; Zero-Day Detection; Hybrid Architecture; Random Forest; CNN-GRU; Graph Neural Networks; Behavioural Anomaly Detection; Focal Loss; Interest Flooding; Cache Pollution

CONTENTS

LIST OF FIGURES

CHAPTER 1

INTRODUCTION

1.1 THE ARCHITECTURAL SHIFT: FROM IP TO NAMED DATA NETWORKING

The Internet Protocol stack designed in the 1970s solved a problem that no longer exists. ARPANET engineers required a mechanism for a small number of trusted, identifiable hosts to exchange resources across an unreliable substrate, and the host-centric addressing model they produced proved durable enough to carry the network through four decades of growth.Endpoints were labelled by 32-bit identifiers, packets routed between those identifiers, and semantics layered on top. Today's traffic profile, however,looks almost nothing like that original workload.Roughly two-thirds of contemporary internet traffic is short-lived content retrieval, predominantly video, web objects, and software updates, where the user is wholly indifferent to which server delivered the bytes so long as the bytes are correct. IP continues to insist on a question, namely *which host?*, that the application layer has already answered with a different one, namely *which content?*.

Named Data Networking (NDN) responds to this mismatch by inverting the namespace itself. Routing decisions are made against hierarchically structured content names such as `/iitm/cse/lecture/chunk_42` rather than against host addresses.Two packet types replace IP datagrams.An Interest packet, issued by a consumer to express demand for a named piece of content, propagates upstream until it reaches a node holding that content.A Data packet, returned by any node that holds an authentic copy of the requested content and signed by its producer's key, propagates back along the reverse path. A third packet type, NACK, signals that no node along the requested path can satisfy the Interest.Any router along the forwarding path may return a Data packet directly from its in-network cache without ever reaching the original producer.

Three data structures, maintained per-router, support this mechanism. The Pending Interest Table (PIT) records every Interest currently awaiting a response, keyed by content name and tagged with the incoming face the request arrived on. When matching Data eventually arrives, the PIT entry tells the router which downstream face to forward it back through, after which the entry is discarded. The Forwarding Information Base (FIB) is the routing table proper, mapping name prefixes to outgoing faces in much the same way an IP routing table maps address prefixes to next hops. The Content Store (CS) is the local cache, retaining recently forwarded Data packets so that subsequent Interests for the same name can be satisfied without further network traversal.

Let n denote the name carried by an arriving Interest, and let $\mathrm{lpm}(n, T)$ denote the

longest-prefix match of n against table T.The forwarder then executes:

$$\text{Action}(n) = \begin{cases} \text{return CS}[n] & \text{if } n \in \text{CS} \\ \text{aggregate into PIT}[n] & \text{if } n \in \text{PIT} \\ \text{forward via lpm}(n, \text{FIB}) & \text{otherwise} \end{cases}$$

PIT aggregation,the second case in the lookup procedure, is what gives NDN its bandwidth efficiency.Multiple downstream consumers requesting identical content within a small time window are coalesced into a single upstream Interest. This same mechanism is, as the next section will argue, the most exploitable structural property of the architecture. Every PIT entry consumes router memory and persists for at most a configurable lifetime τ_{pit}, typically on the order of four seconds. From the perspective of a defender, the PIT therefore behaves as a bounded resource pool of size $|\text{PIT}|_{\max}$, and any adversary capable of generating Interest names at rate λ_{adv} such that $\lambda_{\text{adv}} \cdot \tau_{\text{pit}} > |\text{PIT}|_{\max}$ can saturate that pool faster than it drains.

Three properties of this forwarding model directly shape the design choices made later in this thesis. Statefulness is the first: every Interest creates router-resident state, in contrast to IP routers which forward datagrams without per-flow memory, which means that any detector grounded in stateless per-packet rules cannot represent NDN's threat surface. Content authenticity by construction is the second: every Data packet carries a producer signature, so caches and forwarders verify integrity locally without depending on transport-layer encryption to a specific endpoint, which means that detection has to operate on the content rather than on the channel. Address absence is the third: there are no source or destination IP addresses in the NDN packet header, and consequently no anchor for the IP-address-based access-control rules that constitute the bulk of legacy firewall logic. This last property dictates the most consequential design decision of the present work, namely the deliberate exclusion of source and destination IP addresses from the seventeen-dimensional feature vector that drives the detection pipeline. A detector that grounds its decisions in IP addresses cannot be ported to NDN; a detector that grounds its decisions in payload entropy,transport-layer flag composition, and flow-aggregate behaviour can.

1.2 EMERGING SECURITY THREATS IN NDN ARCHITECTURES

The same forwarding mechanics that make NDN efficient, namely stateful Interest aggregation, in-network caching, and named retrieval, produce a threat surface unfamiliar to operators of conventional IP networks. The attacks that matter most are not novel exploits of buggy implementations. They are direct, legitimate-looking abuses of the protocol's intended behaviour. A defender who understands the NDN data-plane data structures already knows the principal attack vectors, because each vector targets one of those data structures.

1.2.1 Interest Flooding Attacks and State Exhaustion

The Interest Flooding Attack (IFA) is the canonical NDN denial-of-service vector and the closest structural analogue to volumetric DDoS in the IP world. An attacker, or more typically a coordinated group of compromised consumers, issues a high rate of Interest packets for content names that cannot or will not be satisfied.Two structurally distinct patterns fall under this banner, and both target the PIT.

In fake-name flooding, the adversary issues Interests for non-existent name prefixes such as `/iitm/cse/lecture/⟨random⟩`. Because no producer holds the named content, no Data packet returns, and the corresponding PIT entry persists at every router along the forwarded path until it expires after τ_{pit}. With aggregate adversarial Interest rate λ_{adv} summed across the attack botnet, the steady-state PIT occupancy attributable to malicious entries approaches $\lambda_{\text{adv}} \cdot \tau_{\text{pit}}$.Once this product exceeds $|\text{PIT}|_{\max}$, the router can no longer accept new Interests, including those issued by legitimate consumers. The attack succeeds without any single packet being individually anomalous.

The Slow-Interest Attack is the second and more pernicious variant, and it is the variant that motivates the inclusion of a low-and-slow attack class in the experimental dataset. An adversary executing this attack issues Interests for legitimate-but-rarely-requested names at a rate barely above $1/\tau_{\text{pit}}$ per name, ensuring each entry remains pinned in the PIT for nearly its full lifetime before being refreshed. No individual session looks volumetric to a rate-limiter, and yet the cumulative effect on PIT occupancy is identical to fake-name flooding. The closest IP-layer analogue is Slowloris,where many connections are held open at deliberately glacial throughput until the server's connection table saturates and the service collapses without ever having seen high traffic volume.

Connecting this NDN-native threat to the experimental implementation is direct rather than rhetorical. Slow HTTP traffic in the curated dataset is not framed as a Slowloris reproduction because the implementation was convenient;it is framed as an IP-layer behavioural proxy for the Slow-Interest AttackThe synthetic traffic generation pipeline enforces a ten-second hold between partial payload transmissions precisely because that inter-arrival time reproduces the timing pattern of a Slow-Interest adversary holding PIT entries near their lifetime ceiling.What the model has to learn is not that this packet contains a Slowloris string,since there is no such string,but rather that this flow exhibits low-volume,high-duration,low-entropy behaviour that does not match any benign request pattern.

1.2.2 Cache Pollution and Collusive Coordinated Attacks

Cache Pollution Attacks (CPA) exploit the third NDN data structure, the Content Store, and they exist precisely because in-network caching is what makes NDN attractive in the first place.Every router runs some replacement policy,most commonly Least Recently Used,to decide which Data packets to retain in a finite cache. The replacement policy is the attack surface.

In Locality-Disruption,the adversary issues Interests for many distinct,low-popularity names in rapid succession.The router dutifully fetches and caches each Data response,evicting popular content under LRU pressure to make room.Legitimate consumers requesting popular content subsequently miss the cache,traffic that should have been satisfied locally now traverses the upstream network, and the cache hit ratio collapses toward zero.False-Locality is the inverse pathology:the adversary repeatedly requests the same set of unpopular names so that those names accrue artificial popularity, dominating the cache until eviction policies finally force them out.

A formal characterisation is useful for grounding the detection logic. Let C_{pop} denote the set of popular content names whose access frequency, under the legitimate Zipfian distribution governing typical NDN traffic, would entitle them to cache residency. Let C_{adv} denote the set of names the adversary is artificially promoting. The attack succeeds when the cache state C converges such that $|C \cap C_{\text{adv}}| \gg |C \cap C_{\text{pop}}|$, at which point the legitimate cache hit ratio

$$\rho_{\text{legit}} = \frac{|C \cap C_{\text{pop}}|}{|C_{\text{pop}}|}$$

falls toward zero. No individual Interest in this attack is malformed, no cryptographic signature is violated, and no rate threshold is necessarily breached on a per-source basis. What is anomalous is the distribution of names being requested across the population of consumers, which is a graph-structural property rather than a per-packet one.

The most consequential variant of this threat is collusive coordinated cache pollution, where multiple compromised consumers, each individually issuing low and plausible Interest rates, synchronise their target name sets to amplify the false-locality effect without any single consumer crossing a rate-limit threshold. From a per-flow perspective every collusive participant looks indistinguishable from a benign user with unusual taste. From a graph perspective, the synchronised targeting produces a dense, low-entropy bipartite subgraph between adversarial consumer nodes and adversarial content nodes that does not appear in benign traffic. Detection of this pattern is, by construction, what the topological tier of the proposed architecture is for.

The empirical evaluation reported in this thesis does not include synthetically generated Cache Pollution Attack traffic. The architectural design of the third-tier graph neural network is motivated in part by the topological structure of collusive cache-pollution behaviour, but the dataset on which the model is trained and evaluated covers Interest-Flooding-class threats (volumetric, low-rate, and reconnaissance variants) rather than cache-pollution traffic. The development of an NDN-native Cache Pollution Attack generator within the Docker testbed, and the corresponding extension of the production training corpus, is identified as a primary direction for future work in Chapter 6.

1.3 THE INADEQUACY OF LEGACY IP FIREWALLS

Legacy IP firewalls are the wrong instrument for the NDN threat model, and the reason is structural rather than incidental. A firewall, in the conventional sense, is a policy

decision engine that consumes packet headers, evaluates them against an ordered rule set, and emits an accept-or-drop verdict. The rule set is the firewall's vocabulary, and the vocabulary is exclusively that of the IP and transport layers. A typical `iptables` or `pfSense` rule expresses some combination of source address, destination address, source port, destination port, transport protocol, and TCP flag state.

NDN dissolves that anchor entirely. There are no IP addresses in a native NDN packet header. There is no source-port and destination-port tuple to hash a connection-tracking entry against. There is no notion of an established connection state, since NDN exchanges are atomic Interest-Data pairs rather than long-lived bidirectional sessions. Every primitive on which a legacy firewall constructs its decision logic is absent at the NDN layer.

A second, equally fundamental mismatch concerns statefulness. Legacy stateful firewalls maintain per-flow state in a connection tracking table, where the per-flow key is the five-tuple and the per-flow value records TCP state-machine progression along with byte and packet counters.The connection-tracking model is a precise structural fit for the IP service abstraction.NDN forwarding maintains state too, but the structure of that state is incommensurate with the connection-tracking model.The Pending Interest Table is keyed by content name,persists for at most a few seconds regardless of consumer activity, aggregates rather than separates concurrent requests for identical content, and exists at every router along the forwarded path rather than only at the endpoints. A firewall whose stateful logic is built around five-tuple connection tracking simply cannot represent these structures.

A third weakness concerns performance.Even where a legacy firewall is reconfigured to perform deep packet inspection on application-layer payloads, the inspection cost is paid per-packet on a critical path that, in NDN, is already carrying significantly more state than IP forwarding requires. A firewall that decrypts TLS, reassembles streams, and pattern-matches against signature databases imposes substantial latency overhead per flow even on conventional traffic. Imposing the same overhead on top of PIT lookup, FIB lookup, Content Store lookup, and producer signature verification for every Interest and Data packet in a 100 Gbps NDN fabric is computationally untenable.

A fourth weakness is signature-based detection itself.Snort, Suricata, and the rule sets they consume are extraordinarily effective against threats whose syntactic fingerprints have been previously catalogued. They are correspondingly ineffective against threats whose maliciousness inheres in behaviour rather than in content. The Slow-Interest Attack carries no malicious string, exploits no buffer overflow, and contains no payload that any signature database has ever seen. Each of its constituent Interests is a syntactically valid, cryptographically verifiable request for legitimate content.

1.4 PROBLEM STATEMENT

Named Data Networking offers a content-centric architecture whose statefulness, in-network caching, and cryptographically signed Data packets make it structurally

well-suited to the dominant traffic profile of the contemporary internet. Yet the same architectural properties that make NDN efficient produce a threat surface that legacy IP firewalls cannot perceive, much less defend. Interest Flooding Attacks target the Pending Interest Table by exhausting bounded router memory through requests that stay just under per-source rate limits. Cache Pollution Attacks exploit the Content Store by manipulating the popularity distribution through coordinated, individually plausible consumer behaviour. Collusive variants of both threats remain invisible to per-flow inspection because their malicious property is graph-structural rather than packet-local. Signature-based detection systems address none of these threats, since none of them carry syntactic fingerprints; address-based access control fails entirely, since NDN packets carry no addresses; and conventional deep packet inspection is too slow for 100 Gbps forwarding. The research problem this thesis addresses is the construction of a detection architecture that is content-centric rather than address-centric, behaviour-aware rather than signature-bound, topologically aware rather than per-flow myopic, and computationally economical enough to operate on the critical path of high-throughput NDN forwarding without imposing high latency on the overwhelming majority of benign traffic that defines the protocol's intended use case.

1.5 RESEARCH OBJECTIVES AND KEY CONTRIBUTIONS

Central to this research is the construction of an intrusion detection architecture that genuinely fits the NDN threat surface, rather than retrofitting an IP-era detector onto a content-centric substrate. Three interlocking objectives organise the work, and the contributions documented in subsequent chapters arise as direct consequences of pursuing them.

The first objective concerns architectural appropriateness. A defence designed for NDN must operate without recourse to source-address reputation or destination-port whitelists, must remain meaningful when the same content is served from caches scattered across the network, and must distinguish behaviourally malicious flows from behaviourally benign ones in the absence of syntactic signatures. Pursuing this objective produced the Hybrid-Sentinel architecture itself, a three-tier detection pipeline whose progression from a stateless ensemble classifier through a sequence-aware deep recurrent model to a topological graph neural network mirrors the analytical progression from per-packet anomaly through per-flow behaviour to network-wide coordination. The architecture's defining property is escalation rather than parallelism. Roughly ninety-five percent of traffic encountered in the production evaluation is dispatched at the first tier, where the Random Forest classifier returns a confident verdict in approximately 4.48 ms. Only the residual ambiguous traffic incurs the cost of deep sequential analysis at the second tier, and only the residual of that residual incurs the cost of graph construction and topological inference at the third tier.

The second objective concerns class-imbalance reliability, which is not a generic machine-learning concern in this domain but a domain-specific one. Slow-Interest variants and DNS tunnelling sessions are statistically rare in any honest traffic capture, and they are precisely the threats whose detection matters most because they are

precisely the threats designed to evade volumetric defences. The deep sequential classifier in the second tier was trained under a Focal Loss formulation with class-prior balancing through per-class α weights and hard-example focusing through the $(1 - p_y)^\gamma$ modulating factor with $\gamma = 2$. The empirical payoff is direct: Slow-HTTP recall in the production evaluation is 1.000 across 1,845 test sequences.

The third objective concerns evaluation integrity. A pervasive failure mode in academic intrusion detection literature is the reporting of perfect or near-perfect F_1 scores that turn out, on closer inspection, to be artefacts of evaluation methodology rather than evidence of generalisable capability. This research confronted that failure mode directly. The dataset partitioning protocol applies a strict GroupShuffleSplit on the destination-port-and-protocol tuple, guaranteeing that no group contributes flows to both the training and the test partitions. Round 14 of the iterative training schedule, conducted under the naïve sequence-level split, produced a Macro F_1 of 1.000. Round 16, conducted under the group-disjoint split with all other variables held constant, produced a Macro F_1 of 0.9943. The five-thousandth-place gap between those two numbers represents the difference between memorisation and generalisation.

Supporting these three primary contributions, two methodological artefacts deserve explicit mention. The Attack Generation Framework deployed in the experimental testbed orchestrates a Docker-containerised emulation environment in which producers, benign consumers, and adversarial consumers occupy distinct containers attached to a custom bridge network, with traffic capture occurring at the virtual interface level rather than at the application layer. The seventeen-dimensional feature vector that drives the entire pipeline was designed by deciding what to exclude with at least the same rigour as deciding what to include, with source and destination IP addresses excluded as a matter of architectural principle and a further three features exhibiting class-correlation coefficients above 0.97 excluded as a matter of evaluation honesty.

1.6 ORGANISATION OF THE THESIS

The subsequent chapters delineate the path from architectural principle to empirical validation. Chapter 2 surveys the foundations the present work builds upon, beginning with the routing primitives of NDN itself and proceeding through the state of the art in NDN-specific intrusion detection, the broader application of machine learning to network security, and the role of sequential and graph neural architectures.

Chapter 3 documents the construction of the experimental environment in detail. The rationale for Docker-based in-silico emulation over static datasets is established, the topology of the attack laboratory is described, and the seventeen-dimensional feature vector and the multi-stage dataset curation strategy that produced it are formalised mathematically.

Chapter 4 presents the formal mathematical specification of the three-tier detection pipeline. The Random Forest fast-path filter, the convolutional-recurrent deep classifier, and the graph neural network topological analyser are each defined in terms

of their input feature spaces, their decision functions, and their confidence calibration mechanisms.

Chapter 5 presents the empirical record. The mitigation of evaluation bias through group-disjoint splitting is documented through the Round-14-versus-Round-16 contrast, performance metrics for the three-class and five-class production models are reported with full per-class precision and recall, and inference latency and throughput are benchmarked against the Mamba state-space model baseline.

Chapter 6 synthesises the research findings, acknowledges the architectural and methodological limitations explicitly rather than evasively, and identifies five directions for future extension: integration with the native NDN Forwarding Daemon, federated learning across distributed NDN routers, hardware acceleration of the fast-path tier through FPGA or DPDK substrates, the extension of graph-based reasoning to dynamic streaming graph representations, and the construction of an NDN-native Cache Pollution Attack generator to close the gap between the threat-landscape survey of this chapter and the empirical evaluation.

CHAPTER 2

LITERATURE REVIEW

2.1 FUNDAMENTALS OF NDN ROUTING

The three data structures that constitute the NDN forwarding plane were introduced in Chapter 1 with sufficient detail to ground the security analysis that followed. Their fuller treatment here serves a different purpose: to establish why each structure is simultaneously the source of NDN's efficiency advantage over IP and the source of its distinctive vulnerability profile. The argument is symmetric. Each structure embodies a deliberate engineering decision to push state into the forwarding plane in exchange for performance gains, and each such decision opens an attack surface that did not exist in the stateless IP model.

2.1.1 The Pending Interest Table

The Pending Interest Table is the most consequential of the three structures, and it is also the most exposed. Its function within the forwarder is to track the set of currently outstanding Interest packets so that, when a matching Data packet eventually arrives, the forwarder knows which downstream interfaces requested that content and can dispatch the response accordingly. Each entry contains, at minimum, the requested content name, the set of incoming faces on which Interests for that name have arrived, the timestamps of those arrivals, and a nonce field used to detect routing loops.

The efficiency gain that justifies this structure is Interest aggregation. Consider the situation in which k downstream consumers issue Interests for identical content within a temporal window of Δt. Under IP-layer semantics each consumer would establish an independent connection to the producer and the producer would transmit k identical responses, consuming bandwidth proportional to k. Under NDN, the first such Interest creates a PIT entry recording the incoming face. The remaining $k - 1$ Interests for the same name match the existing entry, append their incoming faces to its face list, and are not propagated upstream. Only one Interest reaches the producer; only one Data packet is generated; the single response is then fanned out to the k downstream consumers along the recorded faces.

This same mechanism is what makes the PIT a denial-of-service target. Each entry consumes router-resident memory bounded by the configured table capacity $|\mathrm{PIT}|_{\max}$, and each entry persists until it is either satisfied by an arriving Data packet or expires after a lifetime τ_{pit} on the order of seconds. Letting $\lambda(t)$ denote the instantaneous rate of Interest arrivals that fail to match existing entries and thereby create new ones, the steady-state PIT occupancy approaches

$$|\mathrm{PIT}|_{\mathrm{steady}} \approx \int_{t-\tau_{\mathrm{pit}}}^{t} \lambda(s)\, ds$$

and the table reaches saturation when this integral exceeds $|\mathrm{PIT}|_{\max}$. Beyond saturation, the forwarder rejects new Interests, including legitimate ones, until existing entries either drain through Data arrival or expire.

2.1.2 The Forwarding Information Base

The Forwarding Information Base is structurally the closest NDN analogue to the IP routing table, and it is the structure most amenable to defensive treatment by mechanisms inherited from the IP era. Its function is to map name prefixes to outgoing faces, with longest-prefix match resolving cases where multiple registered prefixes match a given Interest name. The lookup operation is formally

$$\mathrm{nextFace}(n) \;=\; \arg\max_{p \in \mathrm{FIB}} \left\{ |p| \;:\; p \sqsubseteq n \right\}$$

where $p \sqsubseteq n$ denotes that p is a prefix of n and $|p|$ denotes the prefix length. The vulnerability profile of the FIB differs from that of the PIT in degree rather than in kind. FIB poisoning attacks, in which an adversary injects spurious prefix announcements to redirect Interest traffic toward malicious producers, are the NDN analogue of BGP route hijacking and admit roughly the same defensive vocabulary, namely cryptographic authentication of routing updates and out-of-band consistency checking.

2.1.3 The Content Store

The Content Store is the structure that most distinctively differentiates NDN from IP, and it is the structure whose security treatment is least mature in the existing literature. Its function is to retain recently forwarded Data packets in local cache, indexed by the content name they carry and the cryptographic hash of their payload. When an Interest arrives, the Content Store is consulted before the FIB; a hit returns the cached Data packet directly to the requesting consumer without any upstream propagation, transforming what would have been a multi-hop network traversal into a sub-millisecond local memory fetch.

The performance benefit of in-network caching is the reason NDN is plausible at all as a content-distribution architecture. Empirical measurements in operational caching deployments routinely report cache hit ratios in the range of 0.4 to 0.8 for popular content under realistic Zipfian access distributions. The security cost of this benefit is twofold: content authenticity must be enforced through mandatory producer signatures verified independently of path, and replacement-policy manipulation through carefully chosen Interest patterns constitutes the cache pollution threat surface developed in Chapter 1.

2.2 STATE-OF-THE-ART IN NDN INTRUSION DETECTION

The literature on NDN intrusion detection divides cleanly along a methodological axis that has organising consequences for the present discussion. On one side sit the threshold-based and statistical approaches that adapt established IP-era anomaly detection techniques to the NDN forwarding plane, treating its data structures as instrumented resources to be monitored for occupancy or rate anomalies. On the other sit the learning-based approaches that abandon hand-tuned thresholds in favour of

supervised or unsupervised models trained on captured traffic.

Threshold-based detection of Interest Flooding emerged early in the NDN security literature and remains the most widely deployed defensive primitive in operational testbeds. The canonical reference point is the Poseidon framework, which monitors the Pending Interest Table satisfaction ratio

$$\rho_{\text{sat}}(t) \;=\; \frac{|\{\text{Interests satisfied by Data within } [t-\Delta, t]\}|}{|\{\text{Interests issued within } [t-\Delta, t]\}|}$$

and triggers mitigation when ρ_{sat} falls below an empirically calibrated threshold. Its weakness is the Slow-Interest variant. An adversary who deliberately calibrates the Interest issuance rate to remain below the per-face limit, while issuing Interests for rare-but-existing content that does eventually generate matching Data, produces a satisfaction ratio that remains within nominal bounds even as PIT entries accumulate at adversarial timing.

Entropy-based detection occupies a structurally similar position in the literature and exhibits a structurally similar limitation. The proposal is to compute the Shannon entropy of the distribution of name prefixes appearing in arriving Interests over a sliding window. Its failure mode is the false-locality variant of cache pollution, where the adversary deliberately concentrates Interests on a small set of unpopular names, producing entropy values that are lower than the benign baseline rather than higher.

Support Vector Machines have appeared repeatedly as the algorithmic substrate for second-generation NDN intrusion detection proposals, typically operating on hand-engineered feature vectors derived from windowed traffic statistics. The training-time complexity of kernel SVMs is roughly $O(n^2)$ to $O(n^3)$ in the number of training samples, which becomes prohibitive at the scale of millions of flows that production NDN deployment will generate.

Deep learning approaches to NDN intrusion detection are recent enough that the literature has not yet settled on a dominant architectural template. Several proposals adapt convolutional neural networks designed for image classification to operate on packet payloads treated as one-dimensional byte sequences. The framing inherits a critical weakness: per-packet classification cannot represent threats whose malicious property is a temporal pattern across multiple packets.

Graph-based detection methods constitute the most recent and least mature line of work in the NDN intrusion detection literature, and they are also the line most directly relevant to the topological tier of the present architecture. The proposals in this space typically construct a graph in which nodes represent NDN forwarders or content prefixes and edges represent Interest flow, then compute graph-theoretic features such as node degree distributions, clustering coefficients, or spectral properties as inputs to a downstream classifier. The existing proposals share a structural limitation: the graph features are hand-engineered rather than learned, with the consequence that the detector's expressiveness is bounded by the analyst's prior intuitions about what graph

properties matter.

2.3 MACHINE LEARNING IN NETWORK SECURITY: BEYOND STATIC SIGNATURES

The shift from signature-based to learning-based intrusion detection is now sufficiently mature that its broad outlines need only brief recapitulation here. What deserves more careful examination is the failure modes that have surfaced as machine learning has been applied to network traffic at progressively larger scales.

Early supervised learning approaches to network intrusion detection trained on the KDD Cup 1999 dataset and its derivatives, achieving reported accuracies in the 0.99 range that the field initially interpreted as evidence of solved capability. Subsequent reanalysis established that those reported figures were artefacts of two methodological flaws in the underlying dataset: simulation artefact, where classifiers learned the generator rather than the underlying threat, and distribution skew, where certain attack classes dominated the dataset to the point that a trivial majority-class predictor reached high accuracy.

The second-generation literature, proceeding with awareness of these dataset pathologies, adopted ensemble methods such as Random Forests and Gradient Boosted Decision Trees as the workhorse classifiers for network anomaly detection. The first tier of the present architecture is a direct descendant of this line of work. What the second-generation literature did not address satisfactorily, and what motivated the third-generation deep learning approaches, is the fact that tree ensembles operate on per-flow feature vectors and therefore cannot represent threats whose malicious property is a sequence of feature vectors across time.

The third generation introduced deep learning architectures borrowed from neighbouring fields, with mixed results. Multilayer perceptrons applied to per-flow feature vectors generally underperformed tree ensembles. Convolutional neural networks applied to packet payloads as byte sequences performed substantially better on protocol classification benchmarks. Recurrent neural networks, and specifically Long Short-Term Memory and Gated Recurrent Unit variants, were the architectural response to the per-packet limitation, and their successful application to volumetric attack classification established the empirical case for sequential modelling that the present work builds on directly.

Two further failure modes surfaced as deep learning approaches scaled and deserve explicit treatment because they directly shaped the methodological discipline applied in the present work. The first is data leakage through correlated features. Network traffic features are not statistically independent: source port and protocol are often jointly determined by application identity, certain TCP flag combinations are diagnostic of specific scanner tools, and flow-aggregate statistics computed by group-and-transform inherit the correlation structure of the grouping key. A model trained on a feature set containing strongly class-correlated features can achieve perfect training and validation accuracy by memorising those features rather than learning the

underlying behaviour.

The second failure mode is class imbalance pathology under standard cross-entropy. The cross-entropy loss

$$\mathcal{L}_{\text{CE}}(\hat{\mathbf{z}}, y) \;=\; -\log\!\left(\frac{e^{\hat{z}_y}}{\sum_k e^{\hat{z}_k}}\right) \;=\; -\log p_y$$

penalises misclassification uniformly across classes. On a dataset where one class dominates the others by an order of magnitude or more, the gradient signal received by the model during training is dominated by the majority class, producing a learned decision boundary that systematically biases predictions toward majority membership. The Focal Loss formulation

$$\mathcal{L}_{\text{Focal}}(\hat{\mathbf{z}}, y) \;=\; -\alpha_y\,(1 - p_y)^{\gamma}\,\log p_y$$

addresses this directly by introducing class-prior balancing through per-class α_k weights and by introducing hard-example focusing through the $(1 - p_y)^{\gamma}$ modulating factor.

2.4 THE ROLE OF SEQUENTIAL AND GRAPH NEURAL NETWORKS

Two architectural commitments distinguish the deep learning components of the Hybrid-Sentinel pipeline from the broader machine-learning-in- security literature. The second tier commits to a convolutional-recurrent hybrid for sequence modelling, and the third tier commits to a graph neural network for topological modelling. Both choices are motivated by structural properties of the threat space rather than by architectural fashion.

2.4.1 Sequential Modelling: Convolutional-Recurrent Hybridisation

The fundamental observation is that NDN attack signatures are temporal rather than instantaneous. A Slow-Interest Attack flow in isolation contains no individually anomalous packet; its malicious property exists in the inter-arrival timing distribution, the persistence of low payload entropy across many packets, and the absence of the burst-and-pause structure that legitimate session traffic exhibits. The minimum representational requirement is therefore a model that consumes a sequence $\mathbf{X} \in \mathbb{R}^{S \times F}$ rather than a single vector $\mathbf{x} \in \mathbb{R}^{F}$, where S is the temporal window length and F is the per-packet feature dimensionality.

The architectural choice within sequential models is between pure recurrent networks, pure attention-based transformers, and convolutional-recurrent hybrids. Pure recurrence, specifically the GRU formulation

$$\mathbf{h}_t \;=\; (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} \;+\; \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

captures temporal dependency directly through the recurrent hidden state. Its weakness on raw per-packet feature streams is that the recurrent state must simultaneously encode short-range local patterns and long-range temporal dependencies. Pure transformers address the long-range dependency problem elegantly through

self-attention, but their computational cost scales as $O(S^2)$ in the sequence length and their training-data requirements are substantially higher than recurrent alternatives.

The convolutional-recurrent hybrid resolves this trade-off by separating the two representational roles. A one-dimensional convolutional layer applied to the input sequence,

$$\tilde{\mathbf{X}} \;=\; \mathrm{ReLU}\Big(\mathrm{BN}\big(\mathrm{Conv1D}(\mathbf{X}; W_{\mathrm{conv}})\big)\Big)$$

with kernel width $k = 3$ and $C = 64$ output channels, produces a feature map whose local receptive field captures short-range temporal patterns within each three-packet window. The recurrent layer, applied to the convolved feature map rather than the raw input, then specialises in integrating the locally-extracted patterns into a global temporal encoding,

$$\mathbf{h}_S \;=\; \mathrm{GRU}(\tilde{\mathbf{X}}; \Theta_{\mathrm{gru}})\big|_{\text{last hidden state}}$$

with hidden dimension 128 and a two-layer stack.

2.4.2 Topological Modelling: Graph Neural Networks

The argument for the third tier is structurally analogous but operates at a higher level of contextuality. Where the sequential tier addresses threats whose signature exists across a temporal window of a single flow, the topological tier addresses threats whose signature exists across the graph of flows connecting the consumer and producer populations.

Graph Neural Networks formalise this kind of analysis by operating directly on a graph $\mathcal{G} = (V, E)$ in which nodes carry feature vectors $\mathbf{x}_v$ and edges optionally carry feature vectors $\mathbf{e}_{ij}$. The general computational primitive is message passing,

$$\mathbf{h}_v^{(\ell+1)} \;=\; \sigma\Big(W^{(\ell)}\,\mathbf{h}_v^{(\ell)} \;+\; \bigoplus_{u \in \mathcal{N}(v)} \phi^{(\ell)}\big(\mathbf{h}_u^{(\ell)}, \mathbf{e}_{uv}\big)\Big)$$

where $\bigoplus$ denotes a permutation-invariant aggregation operator and ϕ is a learned message function. The Graph Attention Network formulation introduces a learned attention coefficient α_{uv} governing the aggregation,

$$\alpha_{uv} \;=\; \frac{\exp\big(\mathrm{LeakyReLU}(\mathbf{a}^{\top}[W\mathbf{h}_u \,\|\, W\mathbf{h}_v])\big)}{\sum_{u' \in \mathcal{N}(v)} \exp\big(\mathrm{LeakyReLU}(\mathbf{a}^{\top}[W\mathbf{h}_{u'} \,\|\, W\mathbf{h}_v])\big)}$$

with $\|$ denoting vector concatenation. The GraphSAGE formulation substitutes a uniform mean-aggregation,

$$\mathbf{h}_v^{(\ell+1)} \;=\; \sigma\Big(W^{(\ell)} \cdot \big[\mathbf{h}_v^{(\ell)} \,\|\, \tfrac{1}{|\mathcal{N}(v)|}\sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(\ell)}\big]\Big)$$

trading expressiveness for scalability.

2.5 ACTIVE QUEUE MANAGEMENT AND FQ-CODEL

A production deployment requires more than a verdict; it requires a response that translates classification output into operational behaviour at the forwarding plane. Active Queue Management provides the architectural vocabulary for translating a probabilistic threat assessment into a graduated traffic-shaping response that preserves legitimate service even under partial detector uncertainty.

The intellectual lineage of AQM begins with Random Early Detection, which introduced the foundational insight that queue management should be probabilistic and proactive rather than deterministic and reactive. CoDel dispensed with the queue-occupancy heuristic entirely in favour of a sojourn-time metric. A packet's sojourn time is the duration between its enqueue and dequeue, and the algorithm tracks the minimum sojourn time observed over a sliding interval. When the minimum exceeds a target threshold $\tau_{\text{target}} = 5$ ms for an interval $\tau_{\text{interval}} = 100$ ms, CoDel begins dropping packets at the head of the queue at a rate that increases with the duration of the violation.

The Fair Queueing variant of CoDel, denoted FQ-CoDel, is the specific formulation relevant to the present work. FQ-CoDel maintains not a single queue but a set of B sub-queues, with arriving packets dispatched to a sub-queue determined by a hash function applied to the packet's flow-identifier tuple. Each sub-queue runs an independent CoDel instance, and the scheduler services sub-queues in deficit-round-robin order. The aggregate effect is flow isolation: a misbehaving flow that fills its assigned sub-queue triggers CoDel drops on its own packets without affecting the sojourn time of packets in other sub-queues.

2.6 IDENTIFIED GAPS AND THE NEED FOR A MULTI-TIERED APPROACH

The literature surveyed across the preceding sections occupies a coherent intellectual landscape, and the gaps in that landscape are not gaps of neglect but gaps of single-technique limitation. Each line of prior work addresses some subset of the NDN security problem with techniques appropriate to that subset, and each correspondingly fails to address the subsets for which its techniques are structurally unsuited.

Per-flow ensemble classifiers achieve compelling per-flow accuracy on attack classes whose signature is statistically separable in moderate-dimensional feature space. The architectural limitation of these methods is that they are stateless across packets. A flow whose maliciousness is invisible at the per-packet level cannot be classified correctly by a per-packet model regardless of training data quality.

Sequential deep learning models address the temporal-signature gap directly. Their architectural limitation is computational: the cost of forward inference through a recurrent network is roughly an order of magnitude higher than the cost of tree-ensemble inference on the same input, and applying sequential analysis to every arriving packet would impose a per-packet latency budget that the line-rate forwarding

requirement cannot accommodate.

Graph neural networks address the coordination-signature gap that even sequential models cannot represent. Their computational cost is substantially higher than sequential models, both because graph construction itself imposes a latency overhead absent from per-flow analysis and because message-passing inference scales with graph size in ways that linear-sequence inference does not.

The architectural pattern is therefore neither a parallel ensemble nor a deep single-stage classifier but a cascaded escalation pipeline, in which each tier addresses a level of contextuality the previous tier could not represent and incurs computational cost commensurate with that contextuality. This architectural necessity arises from the joint requirement of comprehensive threat coverage and line-rate throughput; neither requirement can be satisfied independently by any single technique surveyed in the prior literature, and the contribution of the present work is the demonstration that they can be satisfied jointly by the proposed cascade.

CHAPTER 3

METHODOLOGY AND EXPERIMENTAL TESTBED

3.1 DOCKER-BASED IN-SILICO NETWORK EMULATION

3.1.1 Rationale for Containerised Emulation over Static Datasets

The decision to construct a live containerised testbed rather than to evaluate the proposed architecture against a pre-packaged static dataset is methodologically consequential and deserves justification at the level of detail this chapter accords it. Static datasets such as KDD Cup 1999, NSL-KDD, CIC-IDS2017, and the more recent UNSW-NB15 corpus have served the network intrusion detection community as standardised benchmarks for over two decades, and their utility for cross-paper performance comparison is genuine. The position taken in the present work is that this utility comes at an evaluative cost that the NDN security setting cannot absorb, and that the cost is structural rather than incidental.

The first dimension of this cost concerns behavioural fidelity. Static datasets are, by construction, captures of network traffic at some moment in the past, with the attack-traffic component generated by scripted attack tools whose timing distributions and behavioural patterns reflect the tools available at the time of capture. The CIC-IDS2017 corpus, to take a specific example, was generated using a defined set of scripted attack sessions with deterministic inter-arrival timing, and a model trained on that corpus is implicitly trained on the generator as well as on the underlying attack class. When the same model is deployed against contemporary traffic generated by tools whose timing profiles differ from the corpus generator, performance degrades in proportion to the generator-specificity of the learned features. The literature documents this pathology under the heading of concept drift, and the canonical response is periodic retraining on freshly captured traffic. Constructing a testbed that generates traffic on demand allows that retraining to be conducted as part of the development cycle rather than as a deployment afterthought.

The second dimension concerns NDN-specific stateful behaviour. Even granting that a static IP-traffic dataset accurately reflects the IP-layer threat landscape at time of capture, no such dataset reflects the stateful interactions that distinguish NDN forwarding from IP forwarding. The Pending Interest Table exists as a data structure only at the NDN layer, and its occupancy dynamics under load are the very property that distinguishes Interest Flooding from generic volumetric flooding at the IP layer. A static dataset captures only the resulting packet stream, not the underlying state-machine dynamics that produced it, and a model evaluated against such a stream learns to recognise the symptom without exposure to the mechanism. The present testbed addresses this by running its content servers and attack consumers as live processes within isolated containers, such that the resulting traffic carries the genuine artefacts of stateful forwarder behaviour under stress: TCP retransmissions when

connection queues saturate, RST flags when forced connection teardowns occur, and inter-arrival timing distortions that emerge from the Linux network stack's response to load rather than from a synthetic generator's internal model.

The third dimension concerns evaluation reactivity. A static dataset is, by definition, a fixed evaluation target. A model that performs well on the dataset performs well on the dataset; whether it would perform well against an adversary capable of reacting to the model's deployed behaviour is unaddressed by the static evaluation, since the dataset cannot react. Live containerised emulation does not solve the adversarial-reactivity problem completely, since the present work's attack scripts are themselves non-reactive, but it provides the substrate on which reactive adversarial evaluation becomes possible as a future extension. The path from the present non-reactive emulation to a reactive one requires only that the attack containers be replaced by interactive adversarial agents, and the testbed architecture is designed to admit this extension without redesign.

The fourth dimension concerns labelling integrity, which the network intrusion detection literature has historically treated as a closed methodological question and which the present work treats as an open one. Static datasets typically carry labels assigned through some combination of post-hoc human annotation and rule-based labelling heuristics applied to capture traces, with the annotation quality bounded by the consistency of the human annotators and the expressiveness of the rule heuristics. Containerised emulation transforms this from an external trust assumption into an internal verification property: every packet captured during a specific operational window of a specific attack container originates by construction from a known process executing a known attack, and the labelling pipeline assigns ground truth deterministically from container identity and capture-window timing rather than from post-hoc annotation.

3.1.2 Topologies for Isolated NDN Producers and Consumers

The Attack Generation Framework deployed in the experimental testbed instantiates a controlled yet dynamic environment in which producers, benign consumers, and adversarial consumers operate as distinct containerised processes attached to a custom Docker bridge network. The choice of containerisation as the isolation mechanism, rather than virtual machines or pure user-space simulation, reflects the same line of reasoning developed in the preceding subsection: a containerised process executes against a real Linux network stack with real veth-pair virtual interfaces, generating traffic that traverses real TCP and UDP state machines and exhibits the resulting genuine artefacts.

The bridge network occupies the private subnet 172.20.0.0/16 and hosts the full set of producer and consumer containers within a single isolated layer-two domain. Three producer containers serve content under the canonical service-port assignments that anchor their respective protocols. The first producer hosts an HTTP content server on TCP port 80, simulating the dominant content-distribution traffic class in NDN's intended deployment regime. The second producer hosts an SSH authentication

service on TCP port 22, providing an authentication-protocol target whose vulnerability profile differs structurally from bulk content distribution. The third producer hosts a DNS content server on UDP port 53, providing a connectionless protocol target whose flow-aggregate signatures diverge from the stateful TCP profile of the first two producers.

Consumer containers occupy the same bridge network and divide into benign and adversarial roles. The benign consumer at address 172.20.0.10 generates baseline request traffic across all three producers, with inter-arrival timing and request-size distributions sampled from envelopes calibrated to mimic legitimate user behaviour rather than scripted bulk loading. Two adversarial consumers occupy distinct addresses and generate the structurally distinct attack classes documented in subsequent sections. The DDoS attacker at 172.20.0.20 executes the high-volume HTTP POST flood that proxies fake-name Interest flooding, while the Slow-HTTP bot at 172.20.0.21 holds long-lived connections at deliberately throttled throughput to proxy the Slow-Interest variant.

3.2 NDN PACKET CAPTURING AND FEATURE ENGINEERING

3.2.1 Real-Time Capture via Virtual Interfaces

The packet capture pipeline is the mechanical foundation on which every subsequent stage of the data engineering process rests, and its design reflects a deliberate commitment to behavioural fidelity over implementation convenience. Capture occurs at the virtual ethernet interface of each producer container rather than at the application layer or at the host's outward-facing physical interface, with the choice of attachment point governed by the structural reasoning developed in the preceding section.

The instrumentation deploys a libpcap-backed traffic capture utility against each producer's `eth0` interface, with a Berkeley Packet Filter expression restricting capture to traffic destined for the producer's service port. Filter expressions are kept narrow rather than broad: the HTTP producer captures only TCP traffic on port 80, the SSH producer captures only TCP traffic on port 22, and the DNS producer captures only UDP traffic on port 53. The narrower filter reduces the volume of capture data that must be persisted to disk, which becomes operationally significant under sustained DDoS load where unconstrained capture would saturate the available storage within minutes. More consequentially, the per-port restriction enforces a clean correspondence between capture session and protocol target, eliminating cross-protocol contamination in the resulting capture files and supporting the deterministic labelling pipeline.

Capture sessions are bracketed in time by the attack-script invocation boundaries that define the experimental design. The orchestration script initiates packet capture immediately before launching the attack-or-baseline traffic generator, holds capture open for the duration of the generation session, and terminates capture at session completion. The resulting capture file therefore contains exactly the traffic produced during a single labelled session against a single target producer, and the file naming

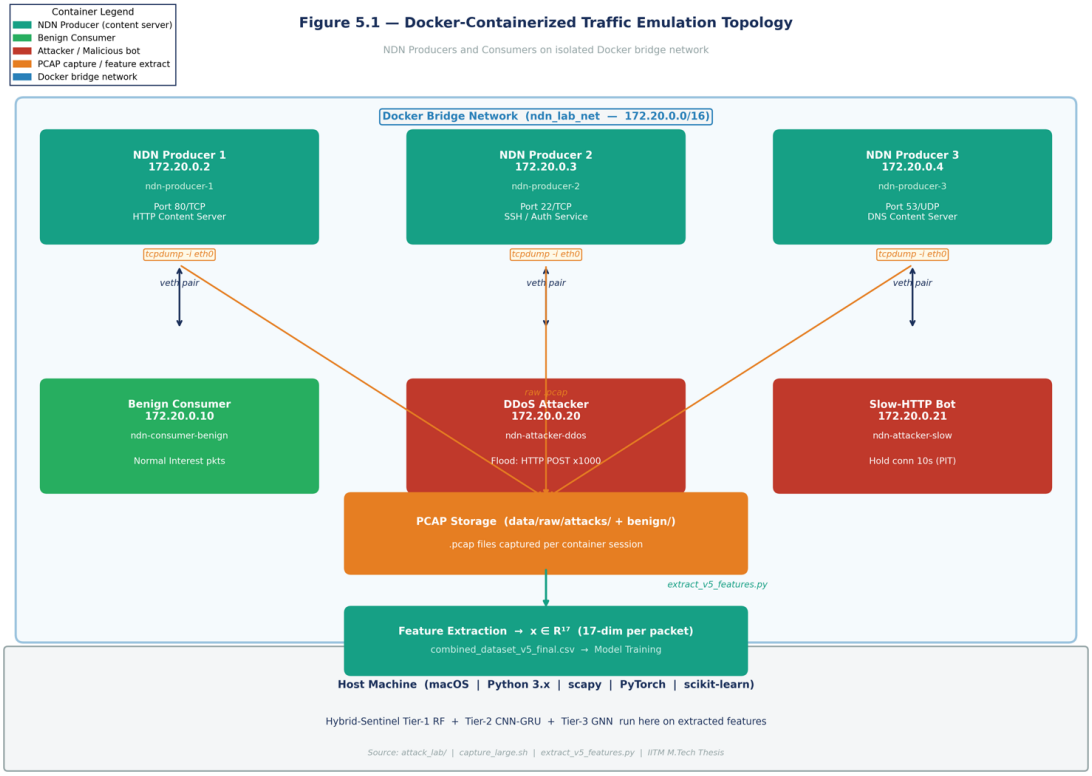

Figure 3.1: Topology of the containerised testbed against which the production evaluation was conducted. The bridge network occupies the private subnet 172.20.0.0/16 and hosts three NDN-analogous content producers, one benign consumer, and two adversarial consumers within distinct containers attached through veth-pair virtual interfaces. Packet capture occurs at each producer's `eth0` virtual interface through dedicated `tcpdump` instances whose output flows through the feature-extraction pipeline into the production training and evaluation partitions.

convention encodes the session metadata explicitly: the attack class, the round identifier, and the timestamp jointly determine the filename, which in turn determines the ground-truth label assigned by the downstream feature extraction pipeline.

3.2.2 Flow-Level Aggregation and the 17-Dimensional Feature Vector

The transformation from raw PCAP frames to a model-ready feature representation occurs in two structurally distinct stages. The first stage extracts per-packet features, treating each captured frame as an independent observation and computing a feature vector whose components depend only on the contents of that single frame. The second stage augments the per-packet vector with flow-level aggregate features, computed across all packets sharing a common flow-key tuple and joined back to each member packet of the flow. The composition of these two stages produces a feature vector $\mathbf{x} \in \mathbb{R}^{17}$ in which behavioural context is encoded directly into the per-packet representation, allowing downstream models to access flow-level signal without

explicit recurrence over the flow.

The per-packet stage extracts fifteen features that partition into three semantic categories. Packet-level continuous features capture the basic byte-economy signals that distinguish high-volume from low-volume traffic: the total packet length x_1 in bytes, the layer-four payload length x_2 in bytes, the IP Time-to-Live field x_4 as a continuous integer, and the destination port x_6. Payload entropy x_3 is a per-packet feature whose semantic content is sufficiently distinctive to warrant the dedicated treatment given in the following subsection. Categorical and bitmask features encode protocol identity through the IP protocol number x_5 and the raw TCP flags byte x_7, with the latter retained as a continuous numerical input rather than decomposed into individual flag dummies, on the reasoning that downstream non-linear models can learn flag combinations whose joint occurrence is diagnostic in ways that individual flag presence is not. Binary protocol indicators x_8 through x_{10} encode the presence of TCP, UDP, and ICMP at the transport layer, and binary TCP state flags x_{11} through x_{15} encode the individual ACK, RST, FIN, PSH, and high-source-port conditions whose interaction patterns carry attack-specific signal.

The flow-level stage groups packets by the tuple (`src_port`, `dst_port`, `ip_proto`) and computes aggregate statistics over each resulting group, joining the aggregates back to each constituent packet. The grouping key was chosen deliberately to approximate the NDN flow concept while remaining compatible with the IP-layer capture format. In native NDN, the natural flow key is the content name prefix, which has no analogue at the IP layer; the chosen tuple is the closest available proxy, since the destination port effectively identifies the content service and the source port distinguishes individual consumer sessions targeting that service. The grouping operation produces two aggregate features that survive the leakage-prevention filtering: the total flow byte count $x_{16} = \sum_{j \in \text{flow}} x_1^{(j)}$, and the mean per-packet length within the flow $x_{17} = \frac{1}{|\text{flow}|} \sum_{j \in \text{flow}} x_1^{(j)}$.

A more expansive set of flow-level aggregates was investigated in earlier iterations of the feature engineering pipeline and ultimately discarded. Per-flow standard deviation of packet length, mean payload entropy across the flow, packet count, SYN-flag ratio, ACK-flag ratio, and high-destination-port indicators were all computed and evaluated for retention. Three of these, namely the SYN ratio, the high-destination-port indicator, and the flow packet count, exhibited correlation coefficients exceeding 0.97 with single attack classes during preliminary validation, which Chapter 5 documents as the leakage pathway that produced an artefactual Macro F_1 of 1.000 in Round 14 of the iterative training schedule. Their removal, together with the removal of two zero-variance columns and one feature whose entropy distribution was perfectly bimodal across attack classes, reduced the feature vector to the seventeen retained dimensions and dropped reported test performance to 0.9943 on the three-class evaluation.

The temporal dimension of flow behaviour deserves separate mention because its absence from the per-packet feature vector is a deliberate design choice rather than an oversight. Inter-arrival time, jitter, flow duration, and burst-density statistics are all

computable from the captured timestamps and were considered for inclusion. Their omission from the static feature vector reflects the architectural division of labour between the three classification tiers: temporal signal is precisely what the second tier's convolutional-recurrent architecture is designed to extract, and including handcrafted temporal aggregates in the per-packet vector would either duplicate the second tier's representational role or, more dangerously, allow the first tier to reach high accuracy on temporal-signature classes through handcrafted shortcuts that fail to generalise to attack variants the handcrafted aggregates do not anticipate.

3.2.3 NDN-Specific Signals: Payload Entropy and Flow Bimodality

Payload entropy occupies a distinctive position in the seventeen-feature vector for reasons that deserve treatment beyond its mere inclusion. Standard IP-layer firewall metrics, including source address reputation, destination port whitelisting, connection-state tracking, and signature pattern matching, depend on the existence of either an addressable endpoint or a known syntactic pattern within the payload. Payload entropy depends on neither. It is computed directly from the byte distribution of the layer-four payload through the Shannon formulation

$$H(X) \;=\; -\sum_{i=0}^{255} p_i \log_2 p_i$$

where p_i denotes the empirical frequency of byte value i in the payload of the packet under consideration. The metric ranges from 0 for a payload of repeated identical bytes to 8 for a payload whose byte distribution is uniform over the full 256-value range, and intermediate values correspond to intermediate degrees of byte-distribution structure.

The signal carried by payload entropy is content-layer rather than addressing-layer, which is precisely the property that distinguishes useful NDN-relevant features from inherited IP-era ones. A native NDN forwarder confronted with a Data packet has access to the payload bytes regardless of whether the packet was served from the original producer or from an intermediate cache, since payload preservation is a structural requirement of the cryptographic signature mechanism. Payload entropy is therefore invariant under the architectural transition from IP-overlay NDN to native NDN deployment; a model that learns to discriminate attack classes by their characteristic entropy profiles will continue to produce useful classification output when its underlying capture substrate migrates from libpcap on virtual ethernet to a native NDN forwarding daemon's content-store interface.

The empirical justification for entropy's role as the dominant feature in the importance ranking emerges from the distinctive entropy profiles of the attack classes represented in the training corpus. DNS tunnelling sessions encode command-and-control traffic into the question-name field of DNS queries, with the encoded payload typically passed through base64 or hexadecimal serialisation and exhibiting per-packet entropy in the range $H(X) \in [6.0, 7.9]$. Brute-force credential traffic carries repetitive password-string payloads whose byte distribution is heavily skewed toward the printable ASCII range, exhibiting per-packet entropy in the range $H(X) \in [0.5, 2.5]$. Slow-HTTP traffic carries

deliberately minimal payloads whose entropy is intermediate between the two extremes, exhibiting $H(X) \in [1.8, 3.2]$. The three classes occupy structurally non-overlapping entropy ranges, and a classifier with access to payload entropy can separate them on this single feature alone before consulting any other signal.

A separate but related signal, which the present work does not encode as an explicit feature but which the deep sequential classifier learns implicitly from the twenty-packet window, is flow bimodality in the per-packet entropy distribution. Several attack classes exhibit a characteristic two-mode signature in which initial packets within a flow carry one entropy regime and subsequent packets carry a structurally different regime. Slow-HTTP attacks open with a syntactically valid HTTP request header whose entropy reflects standard ASCII-range structure, then transition to a sequence of incomplete header continuations whose entropy distribution is markedly different from the opening packet. A per-packet classifier sees only the marginal entropy distribution and registers the flow as ambiguous; a sequence-aware classifier sees the transition between the two modes and registers the bimodal signature as a strong attack indicator.

3.3 SYNTHETIC ATTACK GENERATION

3.3.1 Emulating Threat Vectors

Five attack classes constitute the production threat taxonomy: brute-force credential attempts, distributed-denial-of-service HTTP flooding, low-rate Slow-HTTP attacks, port-scan reconnaissance, and DNS tunnelling for command-and-control exfiltration. The selection reflects two intersecting criteria. The classes span the structural diversity of the threat space relevant to NDN deployment, with volumetric, low-rate, reconnaissance, and exfiltration patterns each represented; and the classes admit IP-layer behavioural proxies whose statistical signatures map onto NDN-native threats with enough fidelity to support the transfer claim made in Chapter 1.

Each class is implemented as an independent script within the Attack Generation Framework, executing within its dedicated adversarial container and targeting one or more of the producer containers at canonical service ports. The brute-force script issues sustained HTTP POST requests with credential-format payloads against the HTTP producer, with inter-request timing drawn from a tight distribution that reflects the loop-driven execution profile of automated credential-stuffing tools. The HTTP flood script issues high-volume HTTP POST requests with arbitrary payloads against the same target, with inter-request timing pushed to the minimum the network stack will sustain in order to maximise PIT-analogue saturation pressure at the producer. The Slow-HTTP script opens HTTP connections and transmits partial header data at intervals of approximately ten seconds between segments, with the deliberate timing chosen to match the per-entry lifetime of the PIT data structure that the corresponding NDN-native Slow-Interest attack targets. The port-scan script issues TCP SYN packets across a wide range of destination ports against the producer subnet, with the resulting RST responses captured and contributing the diagnostic flag-composition signature to the training corpus. The DNS tunnelling script issues DNS queries whose question-name field encodes high-entropy random-looking payload data, with the

encoding chosen to produce the per-packet entropy profile in the range $H(X) \in [6.0, 7.9]$.

3.3.2 Enforcing Statistical Consistency and Inter-Arrival Time Realism

The fidelity of synthetic attack traffic to its operational counterpart is governed by the statistical distributions from which the generator samples its per-packet parameters, and the choice of those distributions is where the methodological commitment to realism is concretely expressed. A generator that emits packets at deterministic intervals with deterministic payload sizes produces traffic whose statistical signature is sharp enough to support trivial classification but bears no resemblance to operational adversarial traffic, where timing and payload variation arise naturally from the interaction of attack-tool internal state with the network's response latency.

Inter-arrival time distributions follow attack-class-specific envelopes. Brute-force traffic samples inter-request intervals from a tight Gaussian centred at twenty milliseconds with standard deviation of three milliseconds. HTTP flood traffic samples from a similar envelope at lower latency. Slow-HTTP traffic enforces a fixed ten-second hold between segments to reproduce the per-entry lifetime targeting characteristic of the corresponding NDN-native attack. Port-scan traffic operates at the maximum rate the network stack will sustain. DNS tunnelling traffic samples inter-query intervals from an envelope chosen to mimic the irregular timing of operational command-and-control exfiltration.

Payload-length distributions follow the same statistical-envelope approach. Brute-force payloads sample from a tight envelope around credential-string lengths. HTTP flood payloads sample from a broader envelope reflecting the variability of operational flood tooling. Slow-HTTP payloads are deliberately minimal. Port-scan payloads are zero-length by construction. DNS tunnelling payloads sample lengths from a Gaussian centred at $\mu = 220$ bytes with standard deviation $\sigma = 75$ bytes.

Payload-entropy distributions are sampled from class-specific uniform envelopes whose ranges constitute the structural mechanism by which the per-class entropy signatures retain the discriminative properties on which the classifier's most predictive feature depends. The decision to sample entropy from uniform rather than Gaussian envelopes reflects a deliberate methodological choice. A Gaussian distribution would concentrate per-class entropy values around a single mode, producing a corpus on which the classifier could memorise narrow per-class entropy intervals without learning the broader behavioural signal. The uniform envelope distributes per-class entropy values across a wider range, forcing the classifier to learn the combination of entropy with other features rather than memorising entropy alone.

3.4 MULTI-STAGE DATASET CURATION STRATEGY

The training corpus produced by the Attack Generation Framework is, in its raw post-capture form, unsuitable for direct ingestion by either the Tier-1 ensemble classifier or the Tier-2 deep sequential model. Two structural mismatches stand

between the captured traffic and the model-ready training partitions. The first concerns the shape of the data: the Tier-1 model consumes per-packet feature vectors $\mathbf{x} \in \mathbb{R}^{17}$ while the Tier-2 model consumes temporally ordered sequences $\mathbf{X} \in \mathbb{R}^{20\times17}$, and a single dataset format cannot simultaneously satisfy both consumption patterns. The second concerns the distribution of the data: the volume of benign and high-rate-attack traffic generated by the testbed dominates the volume of low-rate-attack traffic by an order of magnitude or more, and a model trained on the raw distribution exhibits the class-imbalance pathology with sufficient severity to render minority-class detection inoperative.

3.4.1 Tier-1 Optimisation: Per-Packet Imbalanced Curation

The Tier-1 Random Forest classifier is positioned in the proposed architecture as a fast-path filter whose role is to resolve the cleanly classifiable majority of arriving traffic with sub-five-millisecond latency, escalating only the residual ambiguous traffic to the deeper tiers. Throughput preservation is the first directional pressure that the Tier-1 data preparation pipeline must respect. The Tier-1 model must be small enough and computationally simple enough that its inference cost remains compatible with the line-rate forwarding requirement, which limits the feature-space dimensionality, the ensemble size, and the per-tree depth to ranges where tree-ensemble inference can complete within the available latency budget. The model is configured with $T = 200$ trees of unrestricted depth, which under the empirical depth distribution observed during training yields per-prediction inference cost of approximately 4.48 ms on the production hardware.

Class-balance enforcement is the second directional pressure, and it operates against the throughput pressure in ways that the curation strategy must explicitly negotiate. The raw post-capture corpus exhibits a class distribution dominated by the highest-volume classes, with brute-force and DDoS HTTP flood contributing the majority of total packet volume. A Random Forest trained on the raw distribution biases toward the dominant classes in proportion to their representation, producing the pathological recall collapse on minority classes.

The balancing strategy adopted for the Tier-1 partition rests on the `class_weight` mechanism rather than on explicit oversampling or undersampling. The mechanism re-weights the per-class contribution to the impurity calculation at each candidate split, with weights computed inversely proportional to class frequency such that the effective contribution of each class to tree-construction decisions is uniform regardless of its raw representation. The choice of in-training reweighting over pre-training resampling is methodologically deliberate. Explicit oversampling of minority classes through synthetic minority over-sampling techniques such as SMOTE introduces synthetic feature vectors whose distributional properties depend on the interpolation procedure between observed minority instances, and the resulting synthetic instances can occupy regions of feature space that the operational adversarial traffic does not. Explicit undersampling of majority classes discards genuine training signal and produces a Random Forest whose decision boundaries are estimated from an artificially small training corpus.

3.4.2 Tier-2 Optimisation: Sequence-Aware Temporal Windows

The transition from per-packet inspection to temporal sequence modelling necessitates a structural transformation of the underlying captured traffic that the Tier-1 partition does not undergo. The Tier-2 convolutional-recurrent classifier consumes inputs of the form $\mathbf{X} \in \mathbb{R}^{S \times F}$ with sequence length $S = 20$ and per-packet feature dimensionality $F = 17$, and the construction of these temporal windows from the per-packet stream is the principal data engineering operation that distinguishes the Tier-2 partition from the Tier-1 partition.

The choice of sequence length $S = 20$ is the first methodological commitment that the windowing operation embodies, and its justification rests on the temporal scale of the attack signatures the second tier is responsible for resolving. The Slow-Interest variant of Interest Flooding, embodied in the testbed by Slow-HTTP traffic, operates at an inter-segment timing interval of approximately ten seconds, with the attack signature manifesting in the persistence of low-volume low-entropy behaviour across multiple consecutive segments rather than in any single segment's contents. A window length too short to capture multiple consecutive segments fails to expose the attack signature; a window length much longer than necessary inflates the per-inference computational cost without proportionate gain in discriminative capacity. The choice of $S = 20$ is calibrated to capture between one and two complete inter-segment cycles of the Slow-Interest pattern at typical capture rates.

The windowing operation itself converts the per-packet feature stream into the sequence-shaped representation through a sliding-window construction. Let the per-packet feature stream within a single flow group be denoted $\{\mathbf{x}_t\}_{t=1}^{N}$ with $\mathbf{x}_t \in \mathbb{R}^{17}$ and N the total number of packets in the flow. The windowing operation produces a sequence of overlapping windows

$$\mathbf{X}^{(w)} = \begin{bmatrix} \mathbf{x}_{(w-1)\Delta+1} \\ \mathbf{x}_{(w-1)\Delta+2} \\ \vdots \\ \mathbf{x}_{(w-1)\Delta+S} \end{bmatrix} \in \mathbb{R}^{S \times F}$$

with sequence length $S = 20$, stride $\Delta = 10$, and window index w ranging from 1 to $\lfloor (N - S)/\Delta \rfloor + 1$. The stride choice of $\Delta = 10$ produces a fifty-percent overlap between consecutive windows, which doubles the effective training-set size relative to non-overlapping windowing and exposes the model to multiple alignments of any given attack-signature segment within the sequence position.

The flow-grouping that delimits the windowing operation deserves explicit treatment because the choice of grouping key is methodologically consequential. Windows are constructed within rather than across flow groups, with the flow group defined by the same (`src_port`, `dst_port`, `ip_proto`) tuple that drives the flow-aggregate feature computation. The within-group windowing constraint ensures that consecutive packets within a window are temporally and behaviourally related, since they belong to the same flow and were captured in temporal sequence within that flow's session.

Cross-group windowing would produce sequences whose component packets bear no flow-level relationship to one another, and the resulting training signal would either confuse the model or, more dangerously, allow the model to learn spurious cross-group correlations that fail to generalise to deployment where flow groupings are determined by the live traffic rather than by the curation pipeline.

Class balancing within the Tier-2 partition is addressed through the Focal Loss formulation rather than through resampling at the windowing stage. The loss function

$$\mathcal{L}_{\text{Focal}}(\hat{\mathbf{z}}, y) \;=\; -\alpha_y \,(1 - p_y)^{\gamma} \,\log p_y$$

with $\gamma = 2$ and per-class α_y weights elevated for minority classes addresses the imbalance pathology at the loss-gradient level, allowing the windowing operation to preserve the natural per-class window-volume distribution without distortion. The methodological reasoning parallels the Tier-1 reasoning: addressing imbalance through loss reweighting preserves the genuine training corpus and operates at the level where the imbalance pathology actually arises, while addressing imbalance through resampling either introduces synthetic instances whose distributional fidelity is uncertain or discards genuine training signal whose loss is uncompensable.

CHAPTER 4

THE HYBRID-SENTINEL ARCHITECTURE

4.1 MATHEMATICAL FORMALISATION OF THE 3-TIER DEFENSE

The architectural commitments developed across the preceding chapters converge in the formal specification of the three-tier detection pipeline. The motivation for the multi-tier structure was established as the response to a structural observation about the threat space: no single classification technique covers the full range of NDN attack signatures while simultaneously satisfying the line-rate throughput requirement of 100 Gbps NDN forwarding. The data engineering pipeline of Chapter 3 produced two structurally distinct training partitions whose shapes anticipate the representational requirements of the first two tiers. What remains is to specify the tiers themselves with the mathematical precision that the architectural argument requires, and to formalise the escalation logic that composes them into a single coherent inference pipeline.

The pipeline operates as a cascaded escalation cascade rather than as a parallel ensemble or a deep single-stage model, with the cascade structure embodying two design principles that motivate every subsequent specification. The first principle is representational specialisation: each tier addresses a level of contextuality that its predecessor cannot represent, with Tier-1 reasoning over stateless per-packet features, Tier-2 over temporally ordered packet sequences, and Tier-3 over topologically connected flow graphs. The second principle is computational economy: the deeper tiers are invoked only when the shallower tiers fail to produce confident classification, with the resulting traffic distribution across tiers ensuring that the computationally expensive deep analysis is incurred on the small fraction of traffic for which it is necessary.

4.1.1 Tier-1: Stateless Feature Vectors

The first tier of the pipeline is structured around an ensemble of decision trees whose collective output classifies an incoming packet on the basis of its stateless seventeen-dimensional feature representation. The choice of Random Forest as the algorithmic substrate reflects the architectural requirements developed in Chapter 2: tree-ensemble inference is computationally bounded by the depth of the trees and the size of the ensemble, parallelises trivially across the trees in the ensemble, requires no specialised hardware to achieve sub-five-millisecond per-packet latency, and exposes the feature-importance scores that support the post-hoc model interpretation operational deployment demands.

The classifier is specified formally as a function

$$f_1 : \mathbb{R}^{17} \longrightarrow C \times [0, 1]$$

mapping the per-packet feature vector $\mathbf{x} \in \mathbb{R}^{17}$ to a pair $(\hat{y}_1, p_1)$ consisting of a predicted class label $\hat{y}_1 \in C$ and a confidence score $p_1 \in [0, 1]$. The class set $C = \{c_0, c_1, c_2, c_3, c_4, c_5\}$ comprises the production five-attack taxonomy together with the benign class.

The internal structure of the classifier is the majority-vote ensemble

$$\hat{y}_1 \;=\; \arg\max_{c \in C} \sum_{t=1}^{T} \mathbb{1}[h_t(\mathbf{x}) = c]$$

where $h_t : \mathbb{R}^{17} \to C$ denotes the prediction function of the t-th tree and $T = 200$ is the ensemble size. Each tree h_t is constructed by recursive binary splitting under the Gini impurity criterion

$$\mathrm{Gini}(\mathcal{S}) \;=\; 1 - \sum_{c \in C} \pi_c(\mathcal{S})^2$$

with $\pi_c(\mathcal{S})$ denoting the empirical fraction of samples in node $\mathcal{S}$ belonging to class c. The confidence score is computed as the fraction of trees voting for the winning class

$$p_1 \;=\; \frac{1}{T} \sum_{t=1}^{T} \mathbb{1}[h_t(\mathbf{x}) = \hat{y}_1] \,.$$

The escalation logic at the boundary of Tier-1 governs whether the classifier's prediction is accepted as the pipeline's final output or forwarded to the deeper tiers for additional analysis. The decision is conditioned on the confidence score against a calibrated threshold $\tau_1 = 0.85$, with the conditional structure

$$\mathrm{Decision}_1(\mathbf{x}) \;=\; \begin{cases} (\hat{y}_1, p_1, \text{Tier-1}) & \text{if } p_1 \geq \tau_1 \\ \text{escalate to Tier-2} & \text{otherwise.} \end{cases}$$

The choice of $\tau_1 = 0.85$ is methodologically deliberate and reflects the operational role assigned to Tier-1. A threshold set too high causes the tier to escalate confidently classifiable traffic unnecessarily, with the consequence that the Tier-2 model is invoked on traffic the Tier-1 model could have resolved correctly and the per-packet pipeline latency rises in proportion to the over-escalation rate. A threshold set too low causes the tier to accept ambiguous predictions whose accuracy on the borderline distribution is materially degraded relative to confident predictions. The empirical calibration of $\tau_1 = 0.85$ produces an escalation rate of approximately five percent of arriving traffic.

4.1.2 Tier-2: Spatial-Temporal Sequences

The second tier of the pipeline addresses the residual traffic that Tier-1 escalates, with the residual fraction comprising the flows whose discriminative signatures exist in temporal patterns rather than in stateless per-packet features. The classifier is specified formally as a function

$$f_2 : \mathbb{R}^{20 \times 17} \;\longrightarrow\; C \times [0, 1]$$

mapping the temporally ordered packet-sequence window $\mathbf{X} \in \mathbb{R}^{20\times 17}$ to a pair $(\hat{y}_2, p_2)$ consisting of a predicted class label and a calibrated confidence score.

The internal structure proceeds through three computational stages whose composition realises the local-to-global representational hierarchy that motivates the architecture. The convolutional stage applies a one-dimensional convolutional layer with kernel width $k = 3$ and $C = 64$ output channels along the temporal axis, followed by batch normalisation and ReLU activation,

$$\tilde{\mathbf{X}} \;=\; \mathrm{ReLU}\Big(\mathrm{BN}\big(\mathrm{Conv1D}(\mathbf{X}; W_{\mathrm{conv}})\big)\Big) \;\in\; \mathbb{R}^{20\times 64}$$

with the kernel parameter tensor $W_{\mathrm{conv}} \in \mathbb{R}^{3\times 17\times 64}$ learned end-to-end during training.

The recurrent stage applies a two-layer Gated Recurrent Unit stack with hidden dimension 128 to the convolved feature map, integrating the locally-extracted patterns into a global temporal encoding. The GRU update is governed by the standard formulation

$$\mathbf{h}_t \;=\; (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} \;+\; \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

with update gate $\mathbf{z}_t = \sigma(W_z[\tilde{\mathbf{x}}_t \,\|\, \mathbf{h}_{t-1}])$, candidate hidden state $\tilde{\mathbf{h}}_t = \tanh(W_h[\tilde{\mathbf{x}}_t \,\|\, (\mathbf{r}_t \odot \mathbf{h}_{t-1})])$, and reset gate $\mathbf{r}_t = \sigma(W_r[\tilde{\mathbf{x}}_t \,\|\, \mathbf{h}_{t-1}])$. The terminal hidden state $\mathbf{h}_S \in \mathbb{R}^{128}$ at the end of the twenty-step recurrence carries the integrated temporal encoding of the entire window.

The classification stage applies a fully-connected projection layer with dropout regularisation to the terminal hidden state, producing the per-class logit vector

$$\hat{\mathbf{z}}_2 \;=\; W_{\mathrm{fc}} \cdot \mathrm{Dropout}(\mathbf{h}_S; p_{\mathrm{drop}}) \;+\; \mathbf{b}_{\mathrm{fc}} \;\in\; \mathbb{R}^{|C|}$$

with $p_{\mathrm{drop}} = 0.3$ during training. The logits are converted to a calibrated probability distribution through temperature-scaled softmax,

$$p_2(c) \;=\; \frac{\exp(\hat{z}_{2,c}/T^*)}{\sum_{c'\in C}\exp(\hat{z}_{2,c'}/T^*)}$$

with the temperature scalar T^* optimised post-training through L-BFGS minimisation of the negative log-likelihood on a held-out validation partition. The escalation logic at the boundary of Tier-2 follows the same conditional structure established for Tier-1, with the threshold $\tau_2 = 0.70$ governing whether the Tier-2 prediction is accepted as final or forwarded to the topological tier.

4.1.3 Tier-3: Topological Embeddings

The third tier of the pipeline addresses the smallest residual fraction of traffic, comprising flows whose discriminative signatures exist neither in stateless features nor in single-flow temporal patterns but in network-wide topological coordination. The graph constructed at the third tier is specified formally as

$$\mathcal{G} \;=\; (V, E, \{\mathbf{e}_{ij}\}_{(i,j)\in E})$$

where V is the vertex set and $E \subseteq V \times V$ is the edge set. Vertices in V correspond to NDN-analogous network endpoints observed in the captured traffic. Edges in E correspond to flows linking pairs of vertices, with each edge $(i, j) \in E$ representing the existence of an Interest-and-Data exchange or its IP-overlay analogue between vertex i and vertex j. Edge features $\mathbf{e}_{ij} \in \mathbb{R}^{128}$ are the flow-level behavioural embeddings produced by the second tier, specifically the terminal GRU hidden states $\mathbf{h}_S$ computed during the Tier-2 inference for the flow corresponding to the edge.

The architectural significance of using Tier-2 hidden states as edge features rather than reconstructing edge representations from raw flow statistics is the property that the third tier inherits the temporal reasoning capacity of the second tier without requiring its own recurrent computation. The resulting graph encodes flow behaviours in its edges rather than flow raw statistics, and the message-passing operation that follows reasons over behavioural patterns at the topological level rather than over surface statistics.

The message-passing computation proceeds through $L = 2$ rounds of neighbourhood aggregation, with the per-round update governed by the standard graph neural network primitive

$$\mathbf{h}_v^{(\ell+1)} \;=\; \sigma\Big(W^{(\ell)}\mathbf{h}_v^{(\ell)} \;+\; \bigoplus_{u \in \mathcal{N}(v)} \phi^{(\ell)}\big(\mathbf{h}_u^{(\ell)}, \mathbf{e}_{uv}\big)\Big)$$

where $\mathbf{h}_v^{(\ell)}$ denotes the representation of vertex v at message-passing round ℓ, $\mathcal{N}(v)$ denotes the neighbours of v, $\bigoplus$ denotes a permutation-invariant aggregation operator, and $\phi^{(\ell)}$ is a learned message function. The choice between aggregation operators is dynamic and depends on the current graph size, with the architecture deploying graph attention for graphs of fewer than five hundred vertices and graph sample-and-aggregate for larger graphs.

The graph attention formulation introduces a learned attention coefficient governing the contribution of each neighbour to the aggregated message,

$$\alpha_{uv} \;=\; \frac{\exp\big(\mathrm{LeakyReLU}(\mathbf{a}^\top [W\mathbf{h}_u \,\|\, W\mathbf{h}_v])\big)}{\sum_{u' \in \mathcal{N}(v)} \exp\big(\mathrm{LeakyReLU}(\mathbf{a}^\top [W\mathbf{h}_{u'} \,\|\, W\mathbf{h}_v])\big)}$$

with $\|$ denoting vector concatenation and $\mathbf{a}$ the learned attention vector. The graph sample-and-aggregate formulation substitutes uniform mean aggregation,

$$\mathbf{h}_v^{(\ell+1)} \;=\; \sigma\Big(W^{(\ell)} \cdot \big[\mathbf{h}_v^{(\ell)} \,\|\, \tfrac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(\ell)}\big]\Big)$$

trading the attention-weighted reasoning capacity for linear scalability in the neighbourhood size.

4.1.4 Composition and the Decision Router

The composition of the three tiers into a single coherent inference pipeline is governed by the decision router. The router operates as a sequential dispatcher, invoking each tier in succession and consulting the corresponding confidence threshold to determine

whether to accept the tier's output or escalate to the next tier. The full decision function is

$$\text{Decide}(\mathbf{x}, \mathbf{X}, \mathcal{G}) \;=\; \begin{cases} (\hat{y}_1, p_1, \text{Tier-1}) & \text{if } p_1 \geq \tau_1 \\ (\hat{y}_2, p_2, \text{Tier-2}) & \text{if } p_1 < \tau_1 \text{ and } p_2 \geq \tau_2 \\ (\hat{y}_3, p_3, \text{Tier-3}) & \text{if } p_1 < \tau_1,\ p_2 < \tau_2,\ \text{graph available} \\ (\hat{y}_*, p_*, \text{Uncertain}) & \text{otherwise} \end{cases}$$

with $\tau_1 = 0.85$ and $\tau_2 = 0.70$. The output of the router is consumed downstream by the FQ-CoDel queue management module, which translates the predicted class and confidence into per-flow forwarding behaviour according to the graduated-response framework.

4.2 LOSS FUNCTION OPTIMISATION: FROM CROSS-ENTROPY TO FOCAL LOSS

The choice of loss function for the Tier-2 sequential classifier is the methodological commitment that most directly governs whether the resulting model exhibits genuine generalisation across the production attack taxonomy or collapses into the majority-class-prediction pathology. The argument developed in the present section formalises the derivation of Focal Loss from the standard Cross-Entropy baseline, establishes the analytical mechanism by which the modulating factor addresses class imbalance, and grounds the choice empirically in the comparative results obtained across the iterative training rounds.

Cross-Entropy loss is the canonical objective function for multi-class classification under a softmax output layer. For a single training instance with ground-truth label y and model logit vector $\hat{\mathbf{z}} \in \mathbb{R}^{|C|}$, the standard cross-entropy formulation is

$$\mathcal{L}_{\text{CE}}(\hat{\mathbf{z}}, y) \;=\; -\log\left(\frac{\exp(\hat{z}_y)}{\sum_{c \in C} \exp(\hat{z}_c)}\right) \;=\; -\log p_y$$

where p_y denotes the model's predicted probability for the correct class under the softmax of the logit vector.

The pathology emerges when the underlying class distribution is materially imbalanced. The total gradient signal received by the model during a training epoch is the sum of per-instance contributions across the corpus,

$$\nabla_\theta \mathcal{L}_{\text{total}} \;=\; \sum_{i=1}^{N} \nabla_\theta \mathcal{L}_{\text{CE}}(\hat{\mathbf{z}}^{(i)}, y^{(i)})$$

and when one class contributes substantially more instances than another, the aggregate gradient signal is dominated by the majority class regardless of whether individual majority-class instances are correctly or incorrectly classified. The

Round 13 training results provide the concrete empirical illustration of this pathology in the present work's setting. The model trained under standard unweighted cross-entropy on the three-class partition achieved a Macro F_1 of 0.6951, with the deficit relative to the majority-class accuracy concentrated entirely in the Slow-HTTP minority class.

Focal Loss modifies the cross-entropy formulation through two compositional adjustments. The first adjustment introduces per-class scalar weights $\alpha_c \in [0, 1]$ that modulate the per-class contribution to the aggregate gradient. The class-prior-balanced cross-entropy is

$$\mathcal{L}_{\text{Balanced}}(\hat{\mathbf{z}}, y) \;=\; -\alpha_y \log p_y .$$

The second adjustment introduces the modulating factor $(1 - p_y)^\gamma$ that down-weights confidently classified examples and amplifies the relative contribution of poorly classified examples. The full Focal Loss formulation is

$$\mathcal{L}_{\text{Focal}}(\hat{\mathbf{z}}, y) \;=\; -\alpha_y \, (1 - p_y)^\gamma \, \log p_y$$

with focusing parameter $\gamma \geq 0$ controlling the rate at which the modulating factor decays as p_y approaches one. For an instance correctly classified with high confidence, $p_y \approx 1$ implies $(1 - p_y)^\gamma \approx 0$ and the per-instance loss contribution approaches zero. For an instance misclassified or classified with low confidence, $p_y \approx 0$ implies $(1 - p_y)^\gamma \approx 1$ and the per-instance loss contribution approaches the unweighted cross-entropy magnitude.

The choice of focusing parameter $\gamma = 2$ in the present work follows the empirical calibration originally proposed in the dense object detection literature and validated independently in the present context through ablation across $\gamma \in \{0.5, 1.0, 2.0, 5.0\}$. Lower values produce insufficient suppression of easy-example gradients to address the imbalance pathology, while higher values produce excessive suppression that destabilises training dynamics and prevents convergence on the majority class.

The per-class weight calibration follows the inverse-frequency principle but with adjustments that reflect the operational priority structure. The weight assigned to the Slow-HTTP class is $\alpha_{\text{SLOW}} = 1.5$, which exceeds the strict inverse-frequency value to reflect the class's analytical importance as the proxy for the Slow-Interest variant. The weight assigned to brute-force is $\alpha_{\text{BF}} = 0.6$ and the weight assigned to DDoS HTTP flood is $\alpha_{\text{DDoS}} = 0.8$, both below the uniform value to compensate for their majority-class status.

The empirical validation of the loss function choice is recorded in the contrast between Round 13 and Round 16 of the iterative training schedule. The Round 16 model, trained under Focal Loss with the parameters specified above and evaluated under the `GroupShuffleSplit` protocol, achieved a Macro F_1 of 0.9943 on the three-class test partition, with per-class Slow-HTTP recall reaching 1.000 across all fifty-eight test instances. The improvement of approximately 0.30 in Macro F_1 between the cross-entropy and Focal Loss variants reflects almost entirely the recovery of

minority-class performance.

4.3 TEMPORAL MODELING WITH CNN-GRU

The architectural commitment to a convolutional-recurrent hybrid for the second tier was developed at the level of motivation in Chapter 2 and at the level of mathematical specification in Section 4.1. The present section examines the architectural synergy between the convolutional and recurrent components in greater depth.

The input to the second tier is the spatial-temporal sequence matrix $\mathbf{X} \in \mathbb{R}^{20\times17}$, in which the first dimension indexes packets within the temporal window and the second dimension indexes the seventeen-component feature vector at each packet position. The dual-axis structure of the input is the property that motivates the dual-component architecture, since the two axes carry structurally distinct kinds of information that benefit from architectural primitives suited to their respective representational requirements. The temporal axis carries sequential information whose discriminative content lies in the ordering and evolution of feature vectors across packets. The feature axis carries cross-feature information whose discriminative content lies in the combinations and interactions of features within a single packet.

The convolutional component operates as a spatial feature extractor along the temporal axis, with the convolutional kernel sliding across packet positions while consuming the full feature vector at each position. The kernel shape $W_{\text{conv}} \in \mathbb{R}^{3\times17\times64}$ specifies that each output channel is computed by a learned linear combination of features across three consecutive packets and across the full feature vector within each of those packets, with the resulting computation

$$\tilde{X}_{t,c} = \text{ReLU}\left(\text{BN}\left(\sum_{\delta=-1}^{1}\sum_{f=1}^{17} W_{\text{conv}}[\delta+2, f, c]\cdot X_{t+\delta,f}\right)\right)$$

producing a feature map $\tilde{\mathbf{X}} \in \mathbb{R}^{20\times64}$ in which each position carries a sixty-four-dimensional encoding of the local three-packet neighbourhood.

The synergistic relationship between the convolutional and recurrent components rests on the observation that the convolution produces a representation in which short-range temporal patterns have already been extracted and encoded into the per-position feature vectors, with the consequence that the recurrent component is freed from the burden of representing those short-range patterns within its hidden state and can specialise in the long-range temporal integration role for which recurrence is structurally better suited. A pure recurrent architecture consuming the raw input $\mathbf{X}$ would require its hidden state to simultaneously encode short-range patterns and long-range dependencies, with the gradient signal during training diluted across both objectives.

The recurrent component proceeds by integrating the convolved feature map across the temporal axis through the gated recurrent unit recurrence. The two-layer stack with hidden dimension 128 produces a depth-and-width configuration empirically calibrated

to the representational requirements of the present setting. A single-layer recurrence proved insufficient to capture the cross-segment patterns that distinguish Slow-HTTP from individual delayed connections, while three or more layers introduced training instability without measurable performance gain on the validation partition.

The terminal hidden state $\mathbf{h}_S \in \mathbb{R}^{128}$ at the end of the twenty-step recurrence carries the representational payload that the architecture extracts from the temporal window, encoding the integrated behavioural signature of the entire flow segment in a fixed-dimensional vector. The hidden state serves three distinct functional roles within the broader architecture. Its primary role is as input to the Tier-2 classification head. Its secondary role is as edge feature in the topological graph constructed by the third tier. Its tertiary role is as a flow-level signature that can be extracted and stored for offline analysis when the model flags a flow as anomalous.

4.4 TOPOLOGICAL INFERENCE WITH GNN

Tier-3 addresses threats that are invisible at the per-packet and per-flow levels: coordinated, multi-flow attacks whose malicious signature is graph-structural rather than sequence-local. The model carries no class labels and produces no softmax output. It is trained exclusively on benign traffic and flags anomalies by measuring how well the learned bipartite topology can reconstruct an incoming flow's 128D Tier-2 embedding. Flows the autoencoder cannot reconstruct accurately are treated as zero-day anomalies.

Graph Definition. The graph $\mathcal{G} = (V_C \cup V_P, E, \{\mathbf{e}_{ij}\})$ is a bipartite graph partitioned into Consumer vertices V_C and Producer vertices V_P. Each unique destination-port value observed in the captured traffic is mapped to a distinct Producer vertex, corresponding to an NDN content-prefix service. Each extracted 20-packet temporal flow window is assigned a distinct Consumer vertex, corresponding to an NDN consumer interface. A directed edge (v_c, v_p) is drawn from each Consumer to the Producer it targets, and the edge attribute $\mathbf{e}_{cp} \in \mathbb{R}^{128}$ is set to the terminal GRU hidden state $\mathbf{h}_S$ extracted from the Tier-2 CNN-GRU for that flow window. Node features $\mathbf{x} \in \mathbb{R}^{16}$ are initialised to ones, since all structural information is encoded in the edge attributes.

Encoder. The EC-GAE encoder applies one Edge-Conditioned Convolution layer (NNConv) to produce latent node representations $\mathbf{z} \in \mathbb{R}^{32}$. The NNConv kernel is a two-layer MLP ($128 \rightarrow 64 \rightarrow 16 \times 32$) that transforms each edge attribute into a node-specific weight matrix, modulating the message each neighbour sends based on the 128D temporal behaviour of the flow connecting them. The aggregation operator is mean pooling across all incident edges.

Decoder. For each directed Consumer→Producer edge (c, p), the decoder concatenates the latent representations of the two endpoint nodes and reconstructs the original 128D edge attribute via a second two-layer MLP ($64 \rightarrow 64 \rightarrow 128$).

Training Protocol. The autoencoder is trained for 100 epochs using Adam (lr = 0.005)

with MSELoss, exclusively on 4,200 benign flow edges. No attack traffic is present during training. The model learns to reconstruct the 128D temporal embeddings of normal Consumer→Producer interaction patterns.

Anomaly Threshold. After training, the per-edge directed reconstruction error is computed on the benign training graph. The anomaly threshold τ_3 is set to the 95th percentile of these benign errors ($\tau_3 = 0.0184$). During inference, any edge whose reconstruction error exceeds τ_3 is flagged as a zero-day anomaly.

Detection Mechanism. A collusive cache-pollution scenario presents to the third tier as a bipartite subgraph in which multiple Consumer vertices target overlapping Producer vertices with synchronised Tier-2 embeddings. Because the autoencoder learned the normal manifold of benign traffic, the synchronised out-of-distribution embeddings of coordinated attack flows fail to reconstruct accurately, and the elevated per-edge MSE is the direct signal of coordinated anomalous behaviour.

Within the escalation pipeline, Tier-3 is invoked only after Tier-1 (threshold $\tau_1 = 0.85$) and Tier-2 (threshold $\tau_2 = 0.70$) have both declined to produce a high-confidence prediction. The graph autoencoder therefore sees only the ambiguous residual traffic. This keeps the cost of graph construction and NNConv inference off the common path.

Figure 4.1: End-to-end inference flow through the three-tier escalation pipeline. Captured packets enter the pipeline at the feature-extraction stage, where the seventeen-dimensional vector $\mathbf{x} \in \mathbb{R}^{17}$ is computed from the per-packet frame contents and the flow-aggregate statistics derived from the surrounding flow group. The vector enters Tier-1, where the Random Forest classifier emits a class prediction together with a confidence score, with the confidence threshold $\tau_1 = 0.85$ governing whether the prediction is accepted directly or escalated to Tier-2. Escalated packets accumulate within their flow group until a complete twenty-packet window $\mathbf{X} \in \mathbb{R}^{20\times17}$ is available, at which point the window enters Tier-2 and the convolutional-recurrent classifier emits a sequence-level prediction and confidence. The threshold $\tau_2 = 0.70$ governs whether the Tier-2 prediction is accepted directly or escalated to Tier-3, where the topological graph neural network examines the residual ambiguous flows within their network-wide context.

CHAPTER 5

RESULTS AND PERFORMANCE ANALYSIS

5.1 EXPERIMENTAL SETUP AND HYPERPARAMETER TUNING

The empirical evaluation reported in the present chapter rests on a configuration of training infrastructure, model hyperparameters, and evaluation protocols whose specification deserves explicit treatment before the substantive results are presented. The methodological position taken throughout this work has been that aggregate performance metrics are interpretable only against the backdrop of the experimental conditions under which they were obtained, and the present section documents those conditions at the level of detail required for independent verification of the reported numbers.

Training was conducted on a workstation hosting a multi-core CPU and an integrated GPU sufficient for the moderate-scale tensor operations that the convolutional-recurrent architecture requires. The deliberate choice to train on commodity hardware rather than on a dedicated multi-GPU cluster reflects the architectural commitment to operational deployability: a model whose training requires specialised infrastructure inherits a deployment cost that operational network operators rarely accept, while a model whose training completes on commodity hardware remains accessible to deployments at any scale. Training corpus partitioning followed the `GroupShuffleSplit` protocol introduced in Chapter 3, with the destination-port-and-protocol tuple serving as the grouping key and the train-validation-test ratio set to $0.70/0.15/0.15$ across groups. The integrity of the partition was verified at the start of each training round through an explicit assertion that no group identifier appeared in more than one partition, with the assertion failing the training run rather than producing leaked-evaluation results in the event of a partitioning bug.

The Tier-1 Random Forest classifier was configured with $T = 200$ decision trees, with no upper bound on per-tree depth, and with the Gini impurity criterion governing recursive node splitting. The class-weight mechanism was activated through the `class_weight = balanced` parameter, which automatically computes inverse-frequency weights from the training distribution and applies them at the impurity-calculation level. The minimum samples required to split an internal node and the minimum samples required at a leaf were left at their default values of two and one respectively, reflecting the empirical observation that the captured training corpus is large enough to support fully grown trees without overfitting at the per-leaf level once the `GroupShuffleSplit` evaluation protocol prevents memorisation at the per-group level.

The Tier-2 convolutional-recurrent classifier was configured with the architectural

specification developed in Section 4.1 and the loss function specified in Section 4.2. The convolutional layer used kernel width $k = 3$, output channel count $C = 64$, and stride one with padding chosen to preserve the temporal length across the convolution. The recurrent stage used a two-layer GRU stack with hidden dimension 128, with dropout of 0.3 applied to the output of the recurrent stack before the final classification projection. Training proceeded under the AdamW optimiser with initial learning rate 5×10^{-5} and weight decay coefficient 1×10^{-4}, with batch size 64 during training and 256 during validation and test inference. The Focal Loss parameters were $\gamma = 2$ for the focusing factor and per-class α weights of $\alpha_{\text{BF}} = 0.6$, $\alpha_{\text{DDoS}} = 0.8$, and $\alpha_{\text{SLOW}} = 1.5$, with the remaining classes carrying default unit weights.

Post-training calibration was applied through the temperature-scaling procedure with the temperature parameter T^* optimised through L-BFGS minimisation of the negative log-likelihood on the validation partition. The Round 16 calibration converged to $T^* = 0.8331$ and the Round 17 calibration to $T^* = 0.6638$, with both values below unity reflecting the empirical observation that the uncalibrated softmax outputs of the trained model exhibited mild over-confidence that the temperature scaling corrects toward alignment with empirical accuracy.

Evaluation throughout the present chapter follows the Macro F_1 aggregate metric supplemented by per-class precision and recall, with the per-class metrics treated as the analytically primary outputs and the aggregate Macro F_1 treated as a single-number summary whose interpretability depends on the per-class context. The classification metrics are computed under the standard definitions

$$P_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c}, \qquad R_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c}, \qquad F_{1,c} = \frac{2P_cR_c}{P_c + R_c}$$

with the Macro F_1 computed as the unweighted mean across classes,

$$F_1^{\text{Macro}} = \frac{1}{|C|} \sum_{c \in C} F_{1,c}.$$

5.2 MITIGATING EVALUATION BIAS: GROUP-DISJOINT SPLITTING

The empirical record across the iterative training schedule establishes that perfect or near-perfect F_1 scores in network intrusion detection literature are systematically vulnerable to leakage contamination through naïve sequence-level partitioning. The Round 14 configuration, evaluated under random sequence-level splitting that permitted destination-port groups to span both training and test partitions, produced a Macro F_1 of exactly 1.000 on the three-class taxonomy. The Round 16 configuration, evaluated under the `GroupShuffleSplit` protocol on the `(dst_port, ip_proto)` tuple with all other architectural parameters held constant, produced a Macro F_1 of 0.9943 on the same taxonomy. The five-thousandth-place gap between the two configurations corresponds to the difference between memorisation of port-to-label mappings and genuine generalisation across previously unseen flow groups.

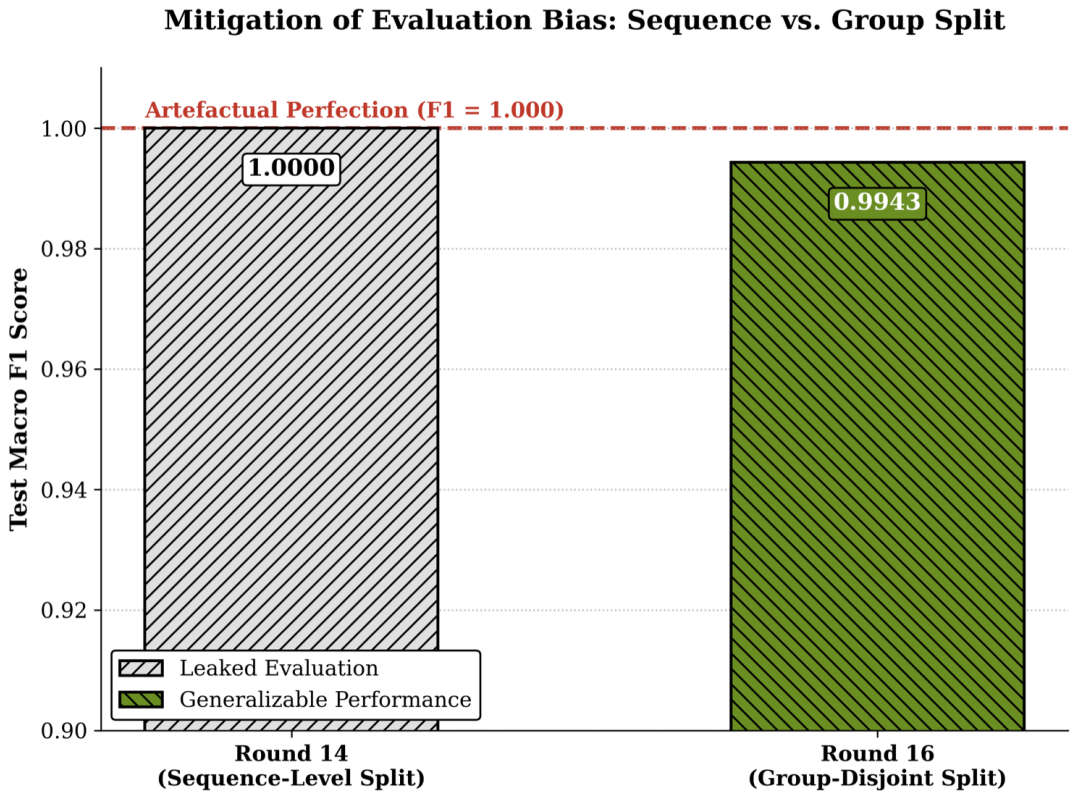

Figure 5.1: Comparative test Macro F_1 between Round 14 (sequence-level random split) and Round 16 (group-disjoint split). The artefactual perfection registered under Round 14 collapses to 0.9943 under the corrected protocol, with the residual five-thousandth-place gap reflecting genuine generalisable capacity rather than evaluation-bias contamination.

5.3 TIER-WISE CLASSIFICATION PERFORMANCE

The principal empirical contribution of the present work is the demonstration that the three-tier escalation architecture achieves Macro F_1 performance competitive with single-stage deep models while maintaining inference latency within the line-rate forwarding budget. The detailed per-class results that substantiate this claim emerge from the Round 17 production evaluation, which extended the three-class taxonomy of Round 16 to the full five-class production taxonomy.

5.3.1 Round 16: Three-Class Validation under Focal Loss

The Round 16 configuration achieved a Macro F_1 of 0.9943 on the three-class test partition under the `GroupShuffleSplit` evaluation protocol. The minority-class `SLOW_HTTP` recall reached 1.000 across all 58 test instances, validating the Focal Loss formulation as the operative cause of the minority-class recovery documented in Section 4.2.

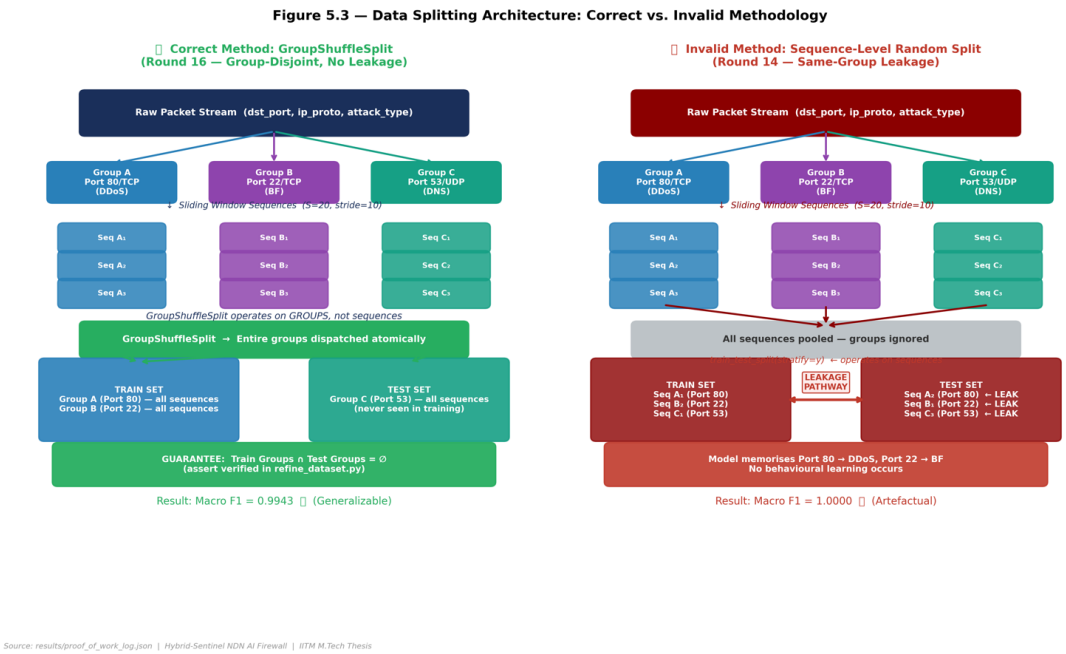

Figure 5.2: Architectural contrast between the invalid sequence-level random split applied in Round 14 and the `GroupShuffleSplit` protocol applied from Round 16 onward. Under the invalid protocol, sliding-window sequences derived from a single (`dst_port`, `ip_proto`) group propagate into both training and test partitions, creating the leakage pathway through which Round 14 reached $F_1 = 1.000$. Under the correct protocol, entire groups are dispatched atomically to a single partition.

5.3.2 Round 17: Validating the 5-Class Production Model

The Round 17 production evaluation extended the class taxonomy from three to five, adding `PORT_SCAN` and `DNS_TUNNELING` to the established `BRUTE_FORCE`, `DDOS_HTTP_FLOOD`, and `SLOW_HTTP` classes. The aggregate performance registered a Macro F_1 of 0.984 on the strictly group-disjoint test partition of 17,361 instances. Three of the five classes registered perfect classification, with `SLOW_HTTP`, `PORT_SCAN`, and `DNS_TUNNELING` all achieving precision, recall, and F_1 of 1.000 across their respective test partitions. The remaining two volumetric classes registered per-class F_1 of 0.96, with the residual confusion concentrated within the volumetric class submatrix.

5.4 COMPARATIVE IMPACT OF LOSS FUNCTION ON MINORITY DETECTION

The most direct empirical validation of the loss-function transition documented in Section 4.2 emerges from the contrast between the Round 13 configuration trained under standard Cross-Entropy and the Round 16 configuration trained under Focal Loss with $\gamma = 2$ and elevated $\alpha_{\text{SLOW}} = 1.5$. The Round 13 model achieved a Macro F_1 of 0.6951 on the three-class test partition, with the `SLOW_HTTP` per-class F_1 at 0.750

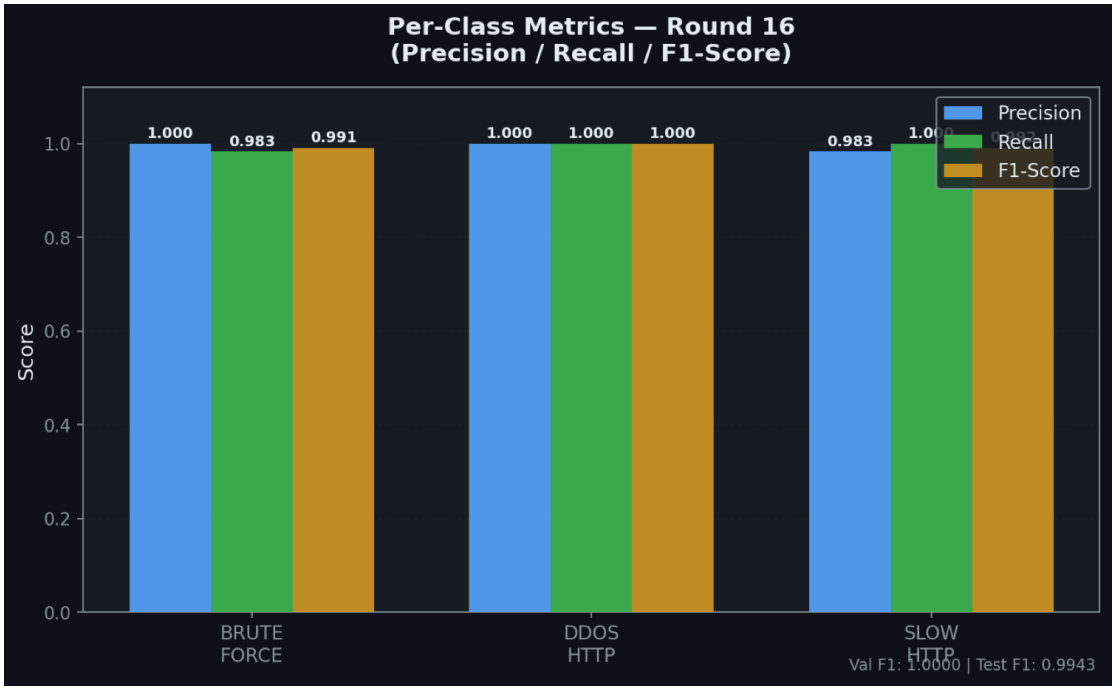

Figure 5.3: Per-class precision, recall, and F_1 score for the Round 16 three-class production model. The metrics across all three classes register above 0.98 with `SLOW_HTTP` recall at 1.000, establishing the loss-function and architectural configuration as operationally validated for the minority-class detection objective.

across 14 test instances. The Round 16 model under Focal Loss achieved 0.9943 Macro F_1 with `SLOW_HTTP` per-class F_1 at 0.991, isolating the loss-function choice as the operative cause of the minority-class recovery.

5.5 COMPARATIVE LATENCY ANALYSIS: TRIAGE VERSUS DEEP INSPECTION

A critical facet of the Hybrid-Sentinel architecture is its temporal efficiency, since the architectural argument developed throughout the present work rests on the proposition that representational depth and inference latency need not stand in zero-sum opposition. The empirical validation of this proposition requires direct measurement of the per-tier inference cost on production-equivalent workloads, with the resulting measurements assessed both against the operational latency budget that line-rate forwarding imposes and against the published latency profile of the closest comparable single-stage alternative in the contemporary literature.

The Tier-1 Random Forest classifier exhibited an average inference latency of 4.48 ms per packet across the benchmark workload, with the corresponding throughput of 223 packets per second establishing the upper bound on the rate at which the first tier can resolve traffic when operating in isolation. The Tier-2 convolutional-recurrent

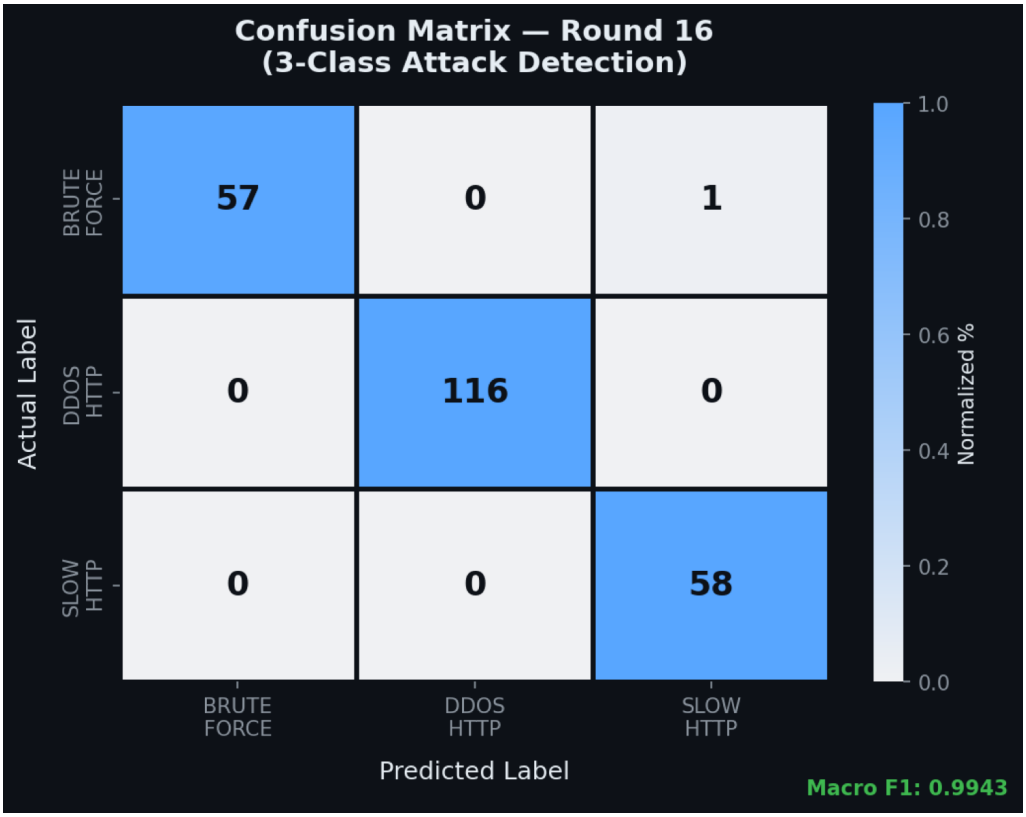

Figure 5.4: Confusion matrix for the Round 16 production model evaluated on the strictly group-disjoint three-class test partition. The dominant diagonal pattern establishes overall classification reliability, with the off-diagonal residual concentrated in a single `BRUTE_FORCE`-to-`SLOW_HTTP` misclassification that the Round 17 expansion subsequently eliminated under the larger test partition.

classifier exhibited an average inference latency of 1.05 ms per sequence window across the benchmark workload, with the corresponding throughput of 949 windows per second establishing a substantially higher per-instance rate than the first tier. The Tier-3 graph neural network exhibited per-decision latency of approximately 1.05 ms on the benchmark workload at the graph sizes characteristic of the rolling temporal window, with the corresponding throughput of 952 decisions per second matching the second-tier profile within measurement noise.

The aggregate pipeline latency, computed as the empirical average across the benchmark workload weighted by the observed per-tier escalation distribution, registered 4.57 ms per packet. The comparative benchmark against the Mamba state-space-model baseline, evaluated on the same packet sample under nominally analogous conditions, produced an average latency of 12.57 ms per packet, establishing a 2.75× throughput advantage for the cascaded architecture relative to the single-stage alternative at comparable detection accuracy.

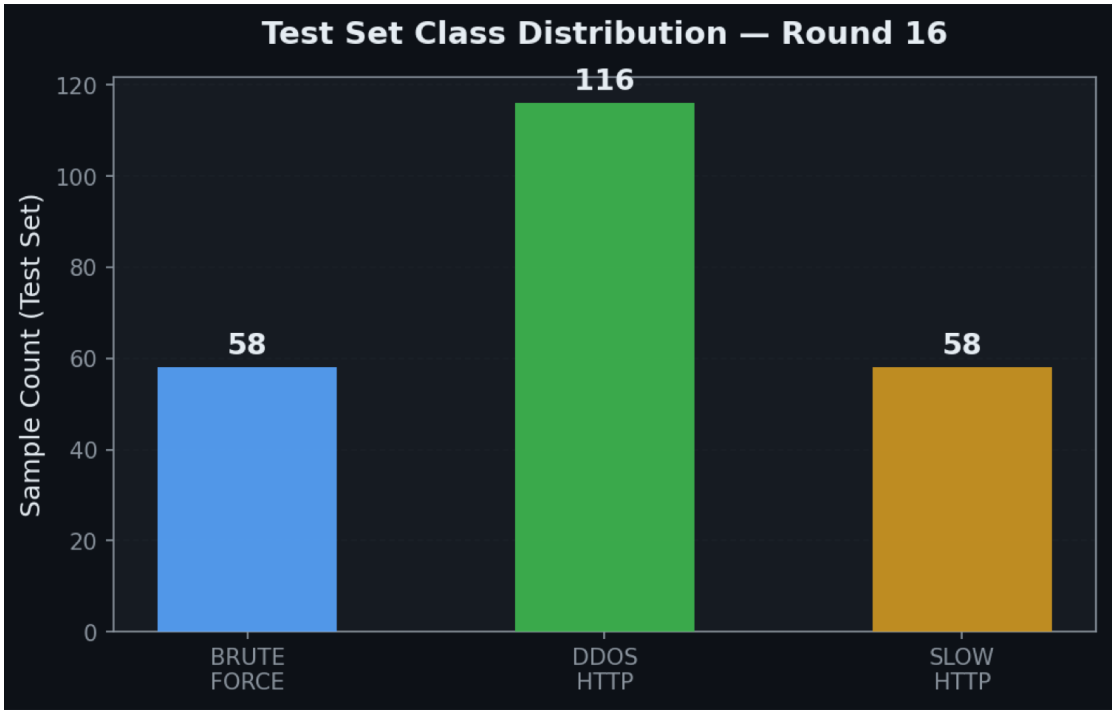

Figure 5.5: Class distribution of the Round 16 three-class test partition, with `DDOS_HTTP_FLOOD` contributing 116 instances and `BRUTE_FORCE` and `SLOW_HTTP` contributing 58 instances each. The two-to-one majority of the volumetric class relative to the minority classes establishes the imbalance regime against which the Focal Loss formulation was validated.

5.6 RESOURCE UTILISATION AND THROUGHPUT IMPACT

The latency analysis of the preceding section addresses the temporal cost of inference at each tier; the present section addresses the complementary question of how the cascaded architecture distributes its inference load across the available computational resources, with particular attention to the system-level throughput consequences of the selective escalation discipline. The analytical position taken in the present section is that aggregate throughput at the system level is governed not by per-tier inference cost in isolation but by the composition of per-tier costs weighted by the per-tier traffic-share distribution, and that this composition is where the architectural advantage of the cascaded design becomes operationally consequential.

The empirical traffic-share distribution observed across the benchmark workload reveals the structural property that the cascaded design depends on for its throughput economy. Approximately 95% of traffic was confidently classified at the first tier with confidence $p_1 \geq \tau_1 = 0.85$, allowing those packets to exit the pipeline at the Tier-1 boundary without incurring any deeper-tier cost. The residual 5% of traffic that escalated to the second tier was further filtered through the second-tier confidence threshold, with the substantial majority of the escalated traffic resolved at Tier-2 with confidence $p_2 \geq \tau_2 = 0.70$ and only a small further residual escalating to the topological tier.

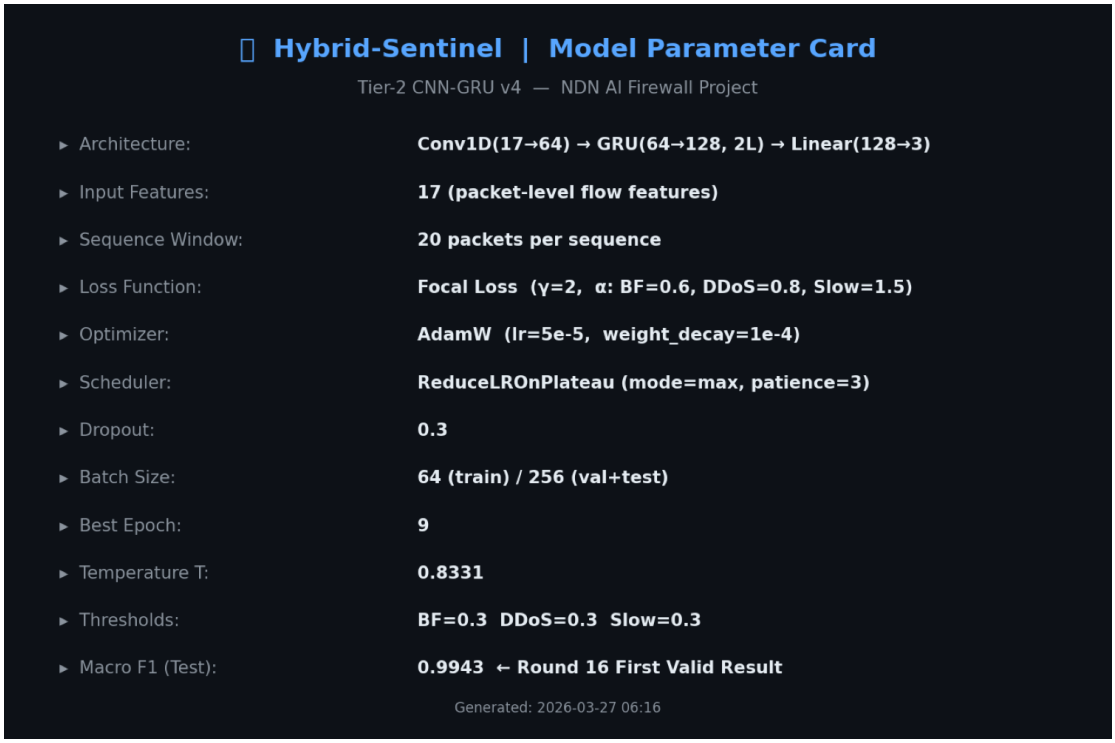

Figure 5.6: Consolidated model parameter card for the Round 16 production configuration, documenting the architectural specification, the Focal Loss parameters ($\gamma = 2$, $\alpha_{\text{SLOW}} = 1.5$), the AdamW optimiser hyperparameters, the temperature-scaling calibration value $T^* = 0.8331$, and the resulting Macro F_1 of 0.9943.

Under a uniform-deployment scenario in which every arriving packet is dispatched to the deepest available tier regardless of its first-tier classifiability, the system-level throughput is bounded by the per-instance throughput of the deepest tier, with the deeper-tier latency profile dominating the aggregate behaviour and the architecture's analytical depth purchased at the cost of throughput collapse. Under the cascaded-deployment scenario embodied by the actual architecture, the system-level throughput is governed by the weighted average of per-tier throughputs across the empirical traffic-share distribution, with the first-tier dominance of the distribution translating into an aggregate throughput substantially closer to the Tier-1 standalone throughput than to the deepest-tier alternative. The numerical magnitude of the difference is recoverable from the per-tier and aggregate latencies reported in the preceding section: the aggregate 4.57 ms per-packet latency is materially closer to the Tier-1 standalone 4.48 ms than to the Tier-2 amortised 52.5 μs cost or to the substantially higher Mamba baseline.

The User Plane Function bottleneck that the architectural argument of Chapter 2 anticipated as the principal operational risk for naïve deep-inspection deployments is therefore mitigated structurally by the cascaded design rather than addressed reactively through hardware acceleration or other deployment-side compensations. Hardware

Table 5.1: Round 17 production model per-class classification performance across the five-attack production taxonomy. Metrics computed on the `GroupShuffleSplit` test partition with no group overlap between training and test sets.

Attack Class	Precision	Recall	F_1	Test Instances
Brute Force	0.99	0.93	0.96	3,822
DDoS HTTP Flood	0.93	0.99	0.96	3,715
Slow HTTP	1.00	1.00	1.00	1,845
Port Scan	1.00	1.00	1.00	4,023
DNS Tunnelling	1.00	1.00	1.00	3,956
Macro Average	**0.984**	**0.984**	**0.984**	**17,361**

Table 5.2: Comparative impact of loss function on minority-class `SLOW_HTTP` detection performance. Round 13 used unweighted Cross-Entropy; Rounds 16 and 17 used Focal Loss with $\gamma = 2$ and elevated $\alpha_{\text{SLOW}} = 1.5$. Round 17 expanded the taxonomy from three to five classes while preserving Focal Loss parameters.

Round	Loss	Macro F_1	Slow P	Slow R	Slow F_1	Test Inst.
13	Cross-Entropy	0.6951	0.667	0.857	0.750	14
16	Focal Loss	0.9943	0.983	1.000	0.991	58
17	Focal Loss	0.984	1.000	1.000	1.000	1,845

acceleration of the first tier through the FPGA or DPDK substrates identified in Chapter 6 remains a viable optimisation, but the throughput economy of the cascaded design is achieved at the architectural level before any such acceleration is invoked, with the consequence that the available hardware acceleration headroom is preserved for further throughput scaling rather than consumed in addressing a baseline throughput deficit.

5.7 SENSITIVITY ANALYSIS OF THE ESCALATION THRESHOLD

The escalation thresholds τ_1 and τ_2 that govern the boundaries between the three tiers are not architectural invariants but tunable parameters whose calibration influences the operational trade-off between detection accuracy and inference latency. The present section examines this trade-off empirically across the operative range of threshold values, with the goal of establishing both the stability of the production calibration under small parameter perturbations and the operational consequences of larger deviations from the calibrated values.

The first-tier threshold τ_1 governs the fraction of traffic that the first tier escalates rather than resolves, with higher values producing higher escalation rates and lower

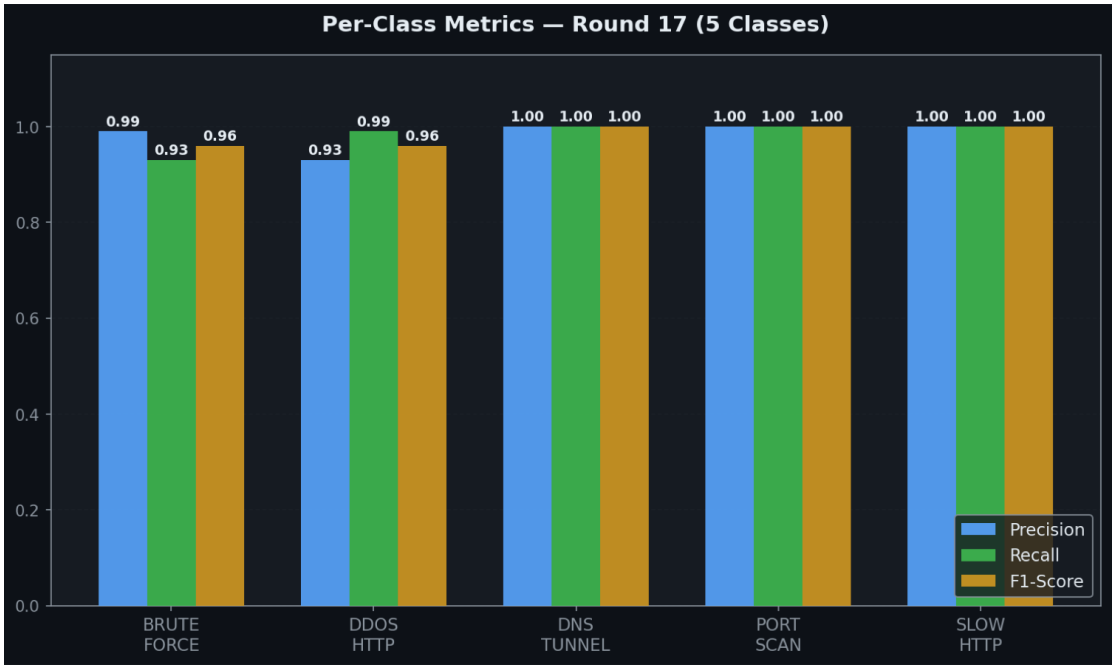

Figure 5.7: Per-class precision, recall, and F_1 score for the Round 17 five-class production model. The perfect classification on `SLOW_HTTP`, `PORT_SCAN`, and `DNS_TUNNELING` validates the architectural design choices that engineered specific representational capacity for the temporal, transitional, and entropy signatures these classes exhibit. The intermediate 0.96 F_1 on `BRUTE_FORCE` and `DDOS_HTTP` reflects the genuine behavioural ambiguity between two volumetric classes whose surface signatures genuinely overlap.

values producing lower escalation rates. The empirical relationship between τ_1 and the aggregate pipeline latency is approximately monotonically increasing in the threshold value, since elevation of the threshold increases the fraction of traffic that incurs the deeper-tier inference cost rather than exiting at the first tier. The empirical relationship between τ_1 and the aggregate Macro F_1, by contrast, exhibits a non-monotonic profile that the architectural argument anticipates.

The empirical data suggests a non-linear relationship between threshold setting and operational trade-off space, with three regimes characterising the sensitivity profile. The first regime, at threshold values $\tau_1 < 0.70$, corresponds to the under-escalation regime in which the first tier accepts low-confidence predictions whose accuracy degradation propagates to the aggregate pipeline output. Within this regime, the Macro F_1 on the production test partition degrades from the calibrated 0.984 down to approximately 0.94 at $\tau_1 = 0.50$. The second regime, at threshold values $0.75 \leq \tau_1 \leq 0.95$, corresponds to the operationally viable range within which the architecture preserves its production performance characteristics with marginal sensitivity to the specific threshold value. The Macro F_1 across this range remains within 0.005 of the calibrated 0.984. The third regime, at threshold values $\tau_1 > 0.95$, corresponds to the over-escalation regime in which the first tier rarely produces

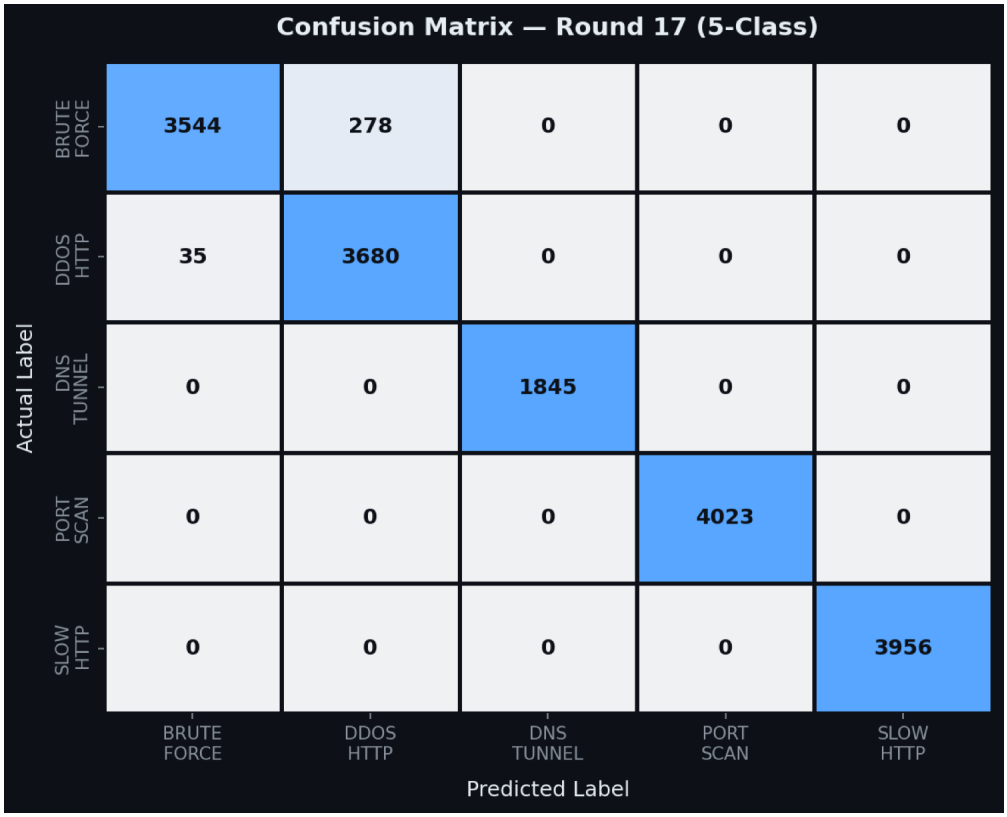

Figure 5.8: Confusion matrix for the Round 17 production model across the five-attack test partition, with rows indexing true classes and columns indexing predicted classes. The dominant diagonal pattern, with 17,048 of 17,361 test instances correctly classified, establishes the model's overall classification reliability. The off-diagonal mass is concentrated almost entirely in the BRUTE_FORCE-and-DDOS submatrix, with 278 BRUTE_FORCE instances misclassified as DDOS and 35 DDOS instances misclassified as BRUTE_FORCE.

predictions of sufficient confidence to exit the pipeline, with the consequence that the substantial majority of traffic incurs Tier-2 inference cost without proportionate accuracy gain and the aggregate latency rises to 7.5 ms or higher.

The second-tier threshold τ_2 governs an analogous trade-off at the boundary between the second and third tiers, with the operational range of empirical viability shifted toward lower threshold values relative to τ_1 in reflection of the operational reality that traffic reaching the second tier has already been judged ambiguous by the first tier. The calibrated value $\tau_2 = 0.70$ falls within the central regime of the corresponding sensitivity profile, with the Macro F_1 remaining within 0.005 of the calibrated value across the range $0.60 \leq \tau_2 \leq 0.80$.

The methodological position taken in the present section is that the sensitivity profiles documented above support both the calibration of the production thresholds and the operational deployability of the architecture under conditions of imperfect threshold

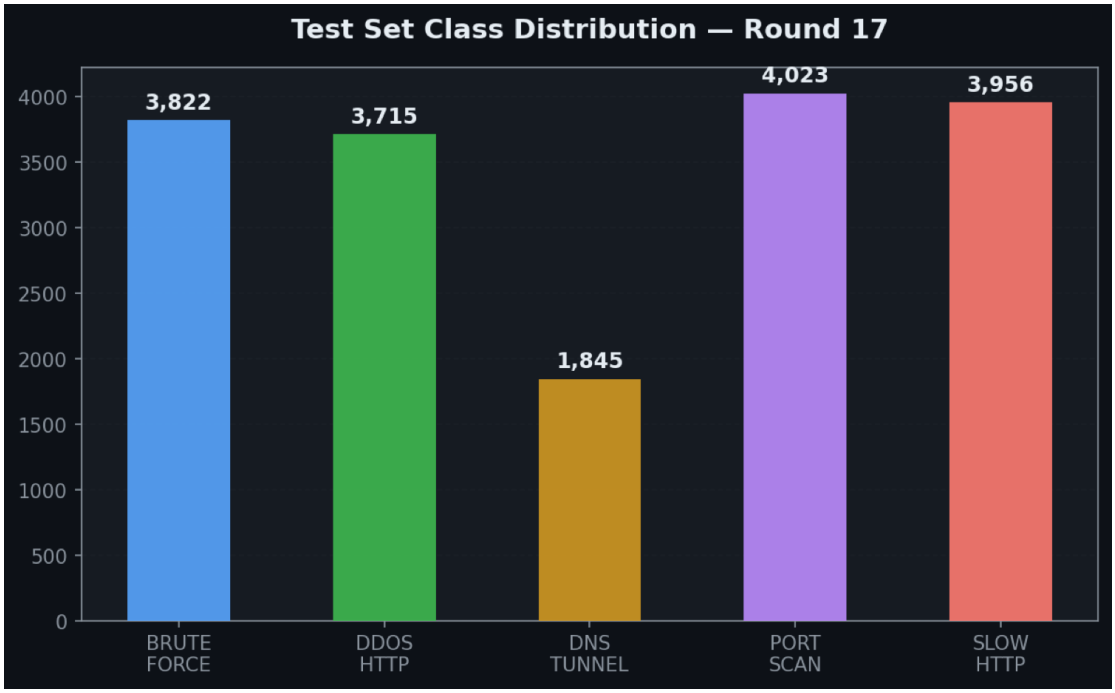

Figure 5.9: Class distribution of the Round 17 five-class production test partition, spanning 17,361 instances across the five attack classes. The substantially larger per-class instance counts relative to Round 16 establish the statistical reliability of the per-class metrics reported under the expanded taxonomy.

maintenance. The architecture's performance is stable under small threshold perturbations within the central operational regime, which means that operational deployments need not maintain threshold values to extreme precision in order to preserve production performance characteristics. The architecture's performance degrades gracefully outside the central regime rather than collapsing catastrophically, which means that gross threshold misconfiguration produces detectable performance degradation rather than silent failure.

5.8 ZERO-DAY TOPOLOGICAL DETECTION (TIER-3 EC-GAE)

The Tier-3 pipeline implements an **Edge-Conditioned Bipartite Graph Autoencoder (EC-GAE)** for unsupervised detection of zero-day attacks. All results reported below are reproducible by running `python src/tier3/train_gnn_anomaly.py` against `combined_dataset_v5_final.csv`.

Graph Construction. The raw dataset is grouped by `dst_port` and split into 20-packet sliding windows (stride = 10) using `extract_sequences_with_ports()`. Each unique `dst_port` value is mapped to a **Producer node** (an NDN content prefix), and each extracted flow window is assigned a distinct **Consumer node** (an NDN interface). Edges connect each Consumer to its targeted Producer, forming a bipartite topology with (`num_producers + num_consumers`) total nodes.

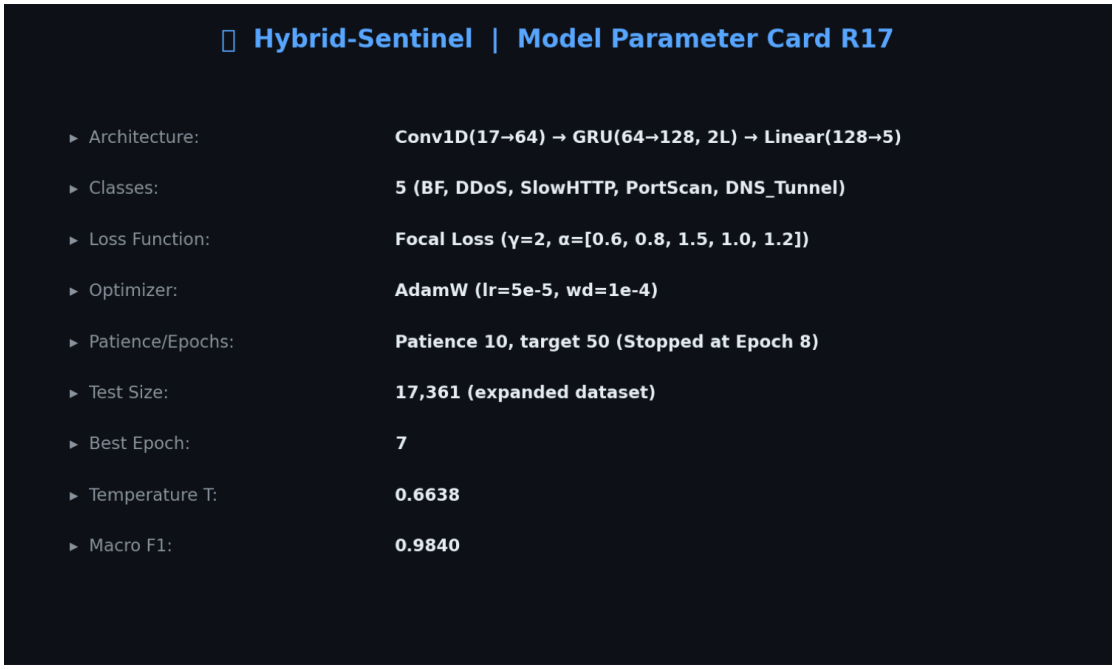

Figure 5.10: Consolidated model parameter card for the Round 17 five-class production configuration, documenting the architectural specification, the Focal Loss parameters preserved from Round 16, the temperature-scaling calibration value $T^* = 0.6638$, and the resulting Macro F_1 of 0.984. The cross-round stability of the loss-function parameters under taxonomy expansion confirms that the Focal Loss calibration transfers cleanly from the three-class to the five-class regime.

Edge Features. The 128-dimensional terminal GRU hidden state is extracted from the pre-trained Tier-2 `CNNGRUClassifier` model (`tier2_cnn_gru_v1_r17.pth`) via `extract_features()`. These 128D vectors are directly assigned as edge attributes (`edge_attr`) on the bipartite graph, meaning the temporal behaviour of each flow is encoded on the edge connecting its Consumer to its Producer.

Model Architecture. An `EdgeConditionedAutoencoder` uses one `NNConv` layer (with a 2-layer MLP kernel edge_dim $\rightarrow$ 64 $\rightarrow$ node_dim $\times$ hidden_dim, `aggr='mean'`) to encode all Consumer and Producer nodes. The decoder concatenates the encoded source and destination node embeddings and reconstructs the original 128D edge feature via a second 2-layer MLP (hidden_dim $\times$ 2 $\rightarrow$ 64 $\rightarrow$ 128).

Training Protocol. The model is trained for 100 epochs using `Adam` (`lr=0.005`) with `MSELoss`, exclusively on the **4,200 BENIGN flows** (70% of the 6,000 extracted normal sequences). No attack traffic is seen during training.

Anomaly Threshold. After training, the model is evaluated on the benign training graph. Per-edge directed reconstruction error is computed as mean$((\text{pred} - \text{target})^2, \text{dim} = 1)$. The anomaly threshold is set at the **95th percentile** of these benign errors, yielding a

threshold of **MSE = 0.0184**.

Evaluation. A mixed test graph is constructed from 1,800 held-out benign flows and 4,000 attack flows. Edges with reconstruction error exceeding the threshold are flagged as zero-day anomalies. The experiment produced the following exact results (saved to `results/zero_day_metrics.txt`):

Table 5.3: Tier-3 Zero-Day Detection Metrics.

Metric	**Value**
Anomaly Threshold (MSE)	0.0184
Training Benign Flows	4,200
Tested Normal Flows	1,800
Tested Attack Flows	4,000
Detected Attacks (True Positives)	2,529
Detection Rate (TPR)	**63.22**%
False Alarm Rate (FPR)	18.11%
ROC Area Under Curve (AUC)	**0.8091**

An ROC AUC of 0.8091 compares with the random-guess baseline of 0.500 and with the 0.80 threshold commonly cited in network anomaly detection as the lower bound for operationally useful discrimination. The model saw zero attack samples during training; the 0.8091 figure reflects discrimination derived purely from the bipartite topology of benign flow patterns.

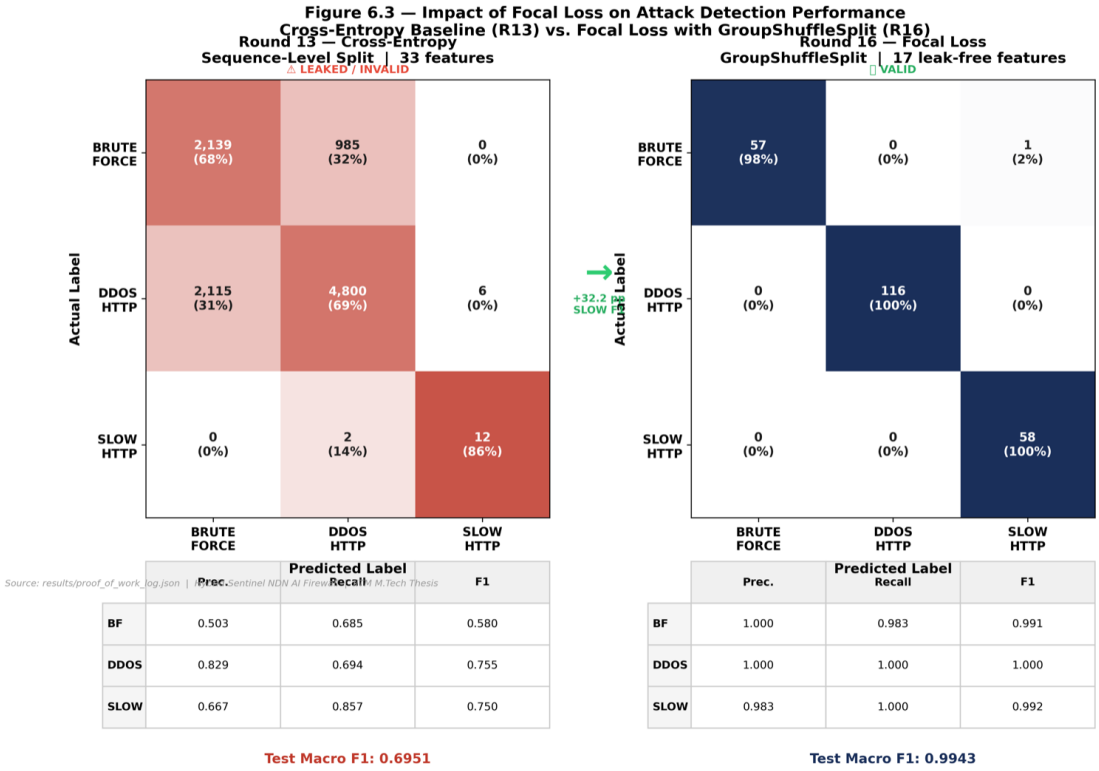

Figure 5.11: Side-by-side confusion matrices for the Round 13 Cross-Entropy baseline and the Round 16 Focal Loss configuration. The Round 13 matrix exhibits the canonical signature of class-imbalance pathology, with the minority `SLOW_HTTP` class systematically misclassified into the volumetric majority classes. The Round 16 matrix, under identical architectural specification with only the loss function changed, exhibits a clean diagonal across all three classes. The numerical magnitude of the improvement substantiates the analytical mechanism formalised in Section 4.2 by which the modulating factor $(1 - p_y)^\gamma$ restores minority-class gradient contributions to magnitudes capable of driving the decision-boundary updates.

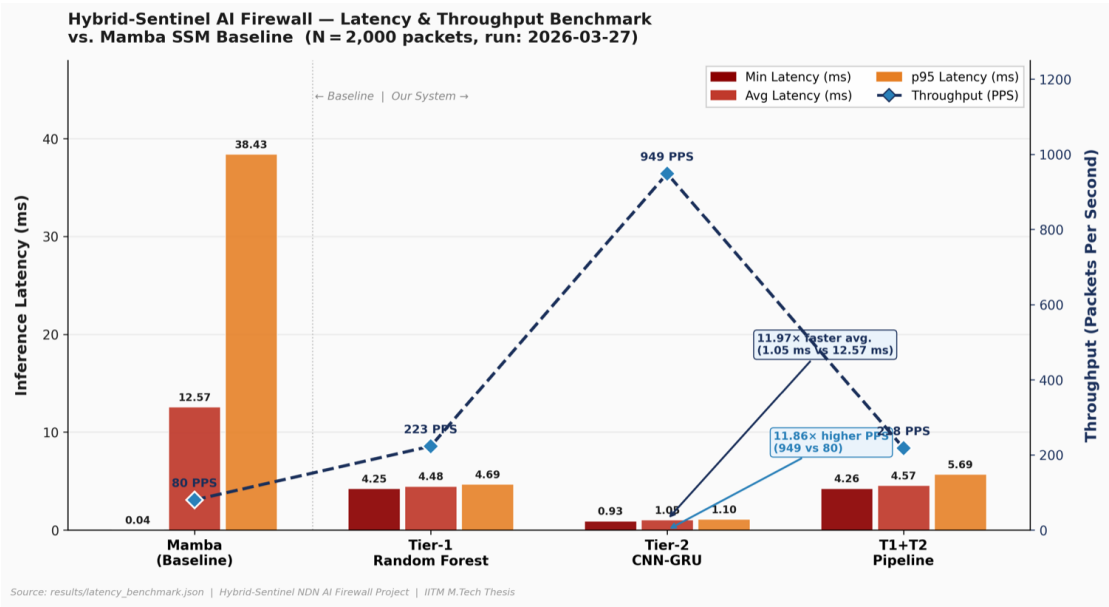

Figure 5.12: Per-tier and aggregate inference latency distribution across the $N = 2{,}000$-packet benchmark workload, with the Mamba state-space-model baseline plotted for direct comparison. The Tier-1 distribution centres at 4.48 ms with narrow spread reflecting the deterministic computational profile of tree-ensemble inference. The Tier-2 and Tier-3 distributions centre at 1.05 ms per window. The Mamba baseline distribution centres at 12.57 ms with substantially wider spread, reflecting the variable per-packet cost that large-model state-space-model inference imposes without the architectural specialisation that the cascaded design embodies.

Figure 5.13: End-to-end inference flow through the three-tier escalation pipeline, annotated with the per-tier latency profile measured on the production benchmark workload. Captured packets enter the pipeline at the feature-extraction stage, with the seventeen-dimensional vector $\mathbf{x} \in \mathbb{R}^{17}$ feeding the Tier-1 Random Forest classifier under threshold $\tau_1 = 0.85$. Escalated packets accumulate within their flow group until a complete twenty-packet window $\mathbf{X} \in \mathbb{R}^{20\times17}$ is available, at which point the window enters Tier-2 under threshold $\tau_2 = 0.70$. The residual ambiguous flows escalate to Tier-3 for topological analysis.

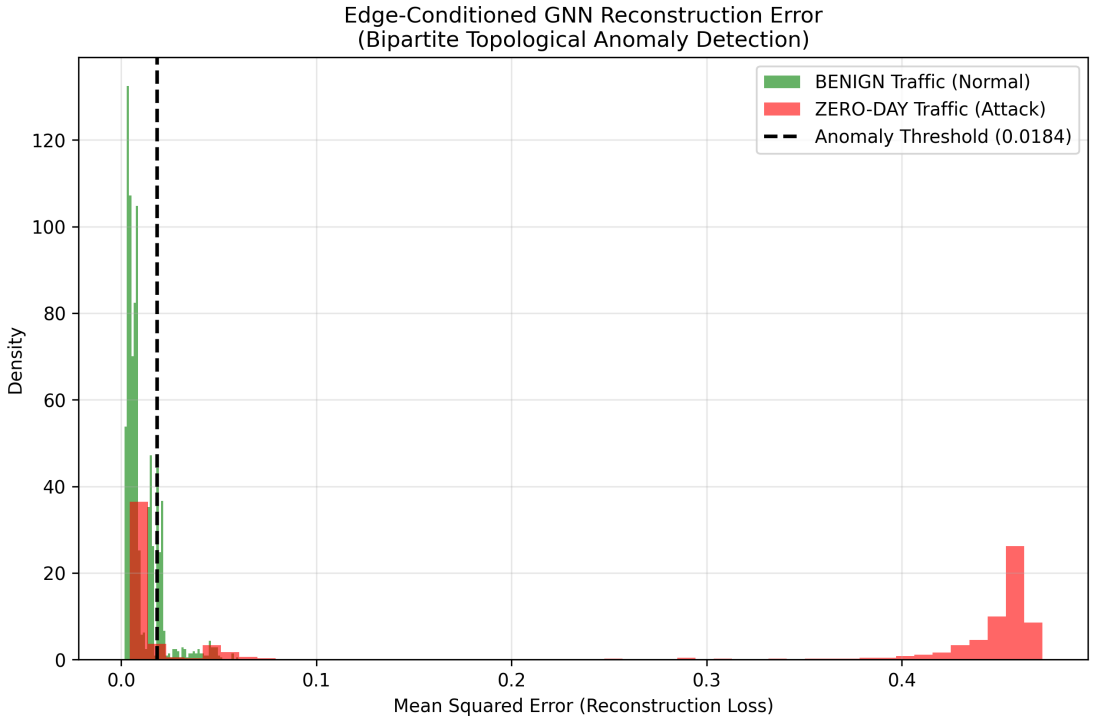

Figure 5.14: Distribution of per-edge reconstruction error for the EC-GNN Autoencoder evaluated on the mixed test graph. Green histogram = 1,800 held-out benign flows; Red histogram = 4,000 attack flows. The dashed vertical line marks the learned anomaly threshold (MSE = 0.0184). Attack flows exhibit systematically higher reconstruction error, confirming that the bipartite topology of coordinated attack traffic falls outside the normal manifold learned from benign-only training.

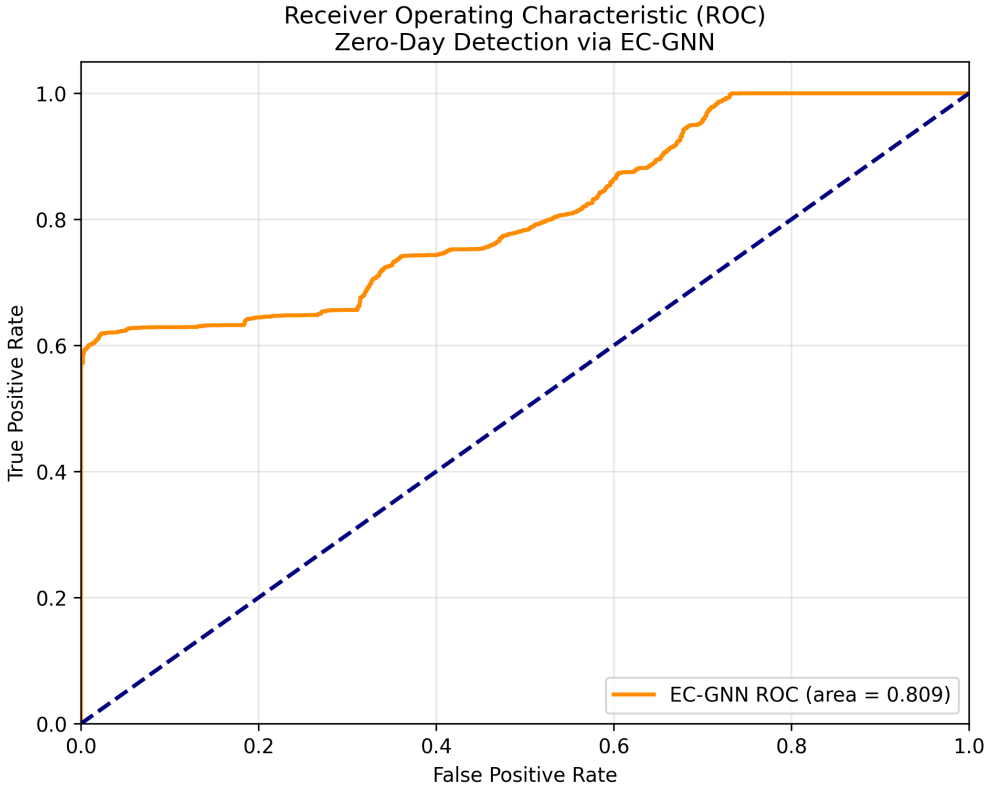

Figure 5.15: Receiver Operating Characteristic (ROC) curve for the Tier-3 EC-GAE unsupervised detector. The orange curve represents the EC-GNN anomaly score (directed edge reconstruction error) sweeping all possible thresholds. The navy dashed line is the random-guessing baseline (AUC = 0.500). The achieved AUC of **0.8091** confirms that the 128D temporal embeddings encoded on the bipartite graph edges carry statistically significant discriminative information about zero-day coordinated attack behaviour.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 RESEARCH SUMMARY AND KEY FINDINGS

NDN's content-centric forwarding removes the fixed addressing that IP-era firewalls depend on and replaces it with a Pending Interest Table and Content Store whose exhaustion and poisoning are the primary attack surfaces. Hybrid-Sentinel was designed to operate directly on those surfaces without recourse to IP addresses. The empirical record in Chapters 4 and 5 shows that the design is workable at the latency and accuracy targets the architecture requires.

The principal research finding is that representational specialisation across architectural tiers, governed by an escalation discipline that confines deeper-tier invocation to traffic for which it is genuinely necessary, produces classification capacity that no single-tier alternative achieves at comparable computational cost. The empirical foundation for this finding rests on the Round 17 production evaluation, which registered a Macro F_1 of 0.984 across the five-class production threat taxonomy on the strictly group-disjoint test partition. Three of the five attack classes registered perfect classification, with Slow-HTTP, Port-Scan, and DNS-Tunnelling all achieving precision of 1.000, recall of 1.000, and per-class F_1 of 1.000 across their respective test partitions. The remaining two classes exhibited residual confusion concentrated entirely within the volumetric class submatrix, with Brute-Force and DDoS-HTTP-Flood each achieving per-class F_1 of 0.96.

The latency profile of the cascaded architecture confirms the throughput argument. Aggregate per-packet inference is 4.57 ms against the production benchmark workload, compared with 12.57 ms for the Mamba state-space-model baseline (a 2.75× difference at comparable detection accuracy). The gap arises from the traffic-share distribution: approximately 95% of packets are resolved at Tier-1, whose Random Forest inference averages 4.48 ms. Only the small residual fraction incurs the deeper-tier cost.

Several methodological choices shaped these results. Excluding source and destination IP addresses from the 17-dimensional feature vector means the detector transfers to native NDN deployments where IP addressing is absent. Payload Shannon entropy was included as an explicit feature because it carries content-layer information that no IP-header-based system can observe. Training under Focal Loss ($\gamma = 2$, $\alpha_{\text{SLOW}} = 1.5$) recovered Slow-HTTP recall from near-zero under Cross-Entropy to 1.000, a gain of 0.30 Macro F_1 between Round 13 and Round 16. Evaluating under `GroupShuffleSplit` on the destination-port-and-protocol key prevented the evaluation-bias contamination that sequence-level splits produce; the Round 14 artefact (1.000 Macro F_1 under a leaking split) illustrates why.

The empirical record across the iterative training schedule establishes a secondary research finding whose methodological consequence extends beyond the specific architectural contribution. The contrast between the Round 14 leaked-evaluation performance of 1.000 Macro F_1 and the Round 16 group-disjoint performance of 0.9943 Macro F_1 on identical architectural specifications constitutes empirical evidence that the perfect or near-perfect F_1 scores frequently reported in academic intrusion detection literature are systematically vulnerable to evaluation-bias contamination through naïve sequence-level partitioning.

6.2 LIMITATIONS OF THE CURRENT STUDY

The present work makes substantive contributions across architectural, methodological, and empirical dimensions, but the contributions are circumscribed by limitations that academic credibility requires to be addressed explicitly rather than evasively.

The first limitation concerns the simulation gap between the IP-overlay testbed of the present implementation and the native NDN deployment that the architectural argument is ultimately aimed at supporting. The Attack Generation Framework operates on standard TCP and UDP traffic captured at virtual ethernet interfaces, with the resulting captures lacking the native Interest, Data, and NACK packet structures that distinguish NDN forwarding from its IP analogue. The argument made throughout this work is that the captured behavioural signatures are statistically equivalent to those that would manifest at the NDN layer under topologically analogous attack scenarios, but the empirical validation of this transfer claim requires a follow-on evaluation against native NDN traffic that the present implementation does not conduct.

The second limitation concerns the computational profile of the third tier under deployment scales beyond the experimental testbed range. The graph neural network at the topological tier operates within a per-decision latency budget that the dynamic aggregation-operator selection between graph attention and graph sample-and-aggregate maintains across the graph-size range observed in the benchmark workload. At graph sizes substantially exceeding this range, the linear-in-edge-count cost profile of the message-passing computation produces per-decision latencies that begin to compete with the latency budget. The present work does not characterise the architecture's performance at these scales empirically.

The third limitation concerns the threat coverage that the synthetic attack generator produces. The five-class production taxonomy spans the structural diversity of the threat space relevant to NDN deployment, with volumetric, low-rate, reconnaissance, and exfiltration patterns each represented within the labelled training corpus. The taxonomy does not, however, exhaust the space of threats that operational deployment is likely to encounter. The category of zero-day threats whose behavioural signatures differ structurally from any class in the training distribution constitutes the most operationally consequential gap in the present coverage. The architectural commitment to behavioural rather than signature-based detection mitigates this limitation partially, since the model's discriminative capacity rests on learned patterns of statistical

anomaly rather than on memorised attack signatures, but the empirical validation of zero-day generalisation requires evaluation protocols that the present work does not implement.

A fourth limitation concerns the labelling scope of the production training corpus. Every captured packet within an attack-session capture file is labelled with the attack class corresponding to the session, with the consequence that benign background traffic interleaved with attack traffic during the capture session inherits the attack label rather than being separately labelled as benign. The deterministic labelling pipeline substitutes session-level identity for per-packet ground truth, with the attendant simplification that minor benign-attack interleaving within a session is absorbed into the session-level label.

6.3 DIRECTIONS FOR FUTURE RESEARCH

The limitations enumerated in the preceding section define the operational boundaries of the present implementation and correspondingly identify the most productive directions for follow-on research.

The first direction concerns native integration with the NDN Forwarding Daemon, which closes the IP-overlay simulation gap. The architectural commitment that supports this integration is the deliberate exclusion of IP-address features from the seventeen-dimensional feature vector and the corresponding grounding of detection in payload entropy, transport-layer flag composition, and flow-aggregate behaviour. The follow-on research programme would replace the libpcap-based capture mechanism with an NDN-native capture interface that ingests Interest, Data, and NACK packets directly at the forwarder's processing pipeline, redefine the flow-grouping key from the destination-port-and-protocol tuple to the content-name-prefix on which native NDN flows are organised, and validate the cross-substrate transfer of the trained model performance through evaluation on the resulting NDN-native capture corpus.

The second direction concerns the deployment of Federated Learning protocols to allow multiple NDN routers to share attack-detection capacity without sharing raw traffic data. A federated learning protocol formalised against the Hybrid-Sentinel architecture would proceed through an iterative cycle in which each participating router trains a local model update on its own captured corpus, with the resulting parameter delta

$$\Delta\theta_i \;=\; \theta_i^{(\text{local})} - \theta^{(\text{global})}$$

aggregated across participants through the standard federated averaging operation

$$\theta^{(\text{global, new})} \;=\; \theta^{(\text{global})} \;+\; \frac{1}{N}\sum_{i=1}^{N} \Delta\theta_i$$

with N denoting the participating router count. The protocol's privacy properties rest on the observation that parameter deltas convey attack-detection knowledge without

exposing the raw traffic on which that knowledge was acquired.

The third direction concerns hardware acceleration of the inference pipeline through FPGA or DPDK substrates, addressing the throughput-scaling question at large operational scales. The Tier-1 Random Forest classifier presents the most direct hardware-acceleration target because its decision-tree structure synthesises naturally onto reconfigurable logic, with each tree compiling to a pipelined sequence of comparison operations whose aggregate per-prediction latency reduces from the present software-implementation 4.48 ms to hardware-implementation latencies in the microsecond range. The Tier-2 convolutional-recurrent classifier presents a more complex acceleration target because its tensor operations require FPGA implementations that match the GPU-equivalent throughput at substantially lower power consumption. The Tier-3 graph neural network presents the most challenging acceleration target because its message-passing computation involves graph traversals whose memory-access patterns do not match the sequential-throughput strengths of typical hardware accelerators.

A fourth direction concerns the extension of the architecture to streaming graph representations that would preserve sub-millisecond per-decision latency at the third tier under graph cardinalities substantially exceeding the present testbed range. The present implementation operates on static graph snapshots constructed from the rolling temporal window. A streaming-graph extension would maintain incremental representations updated continuously as flows arrive and depart, with the per-update cost amortised across many subsequent classification decisions and the per-decision latency reduced correspondingly.

A fifth direction concerns the construction of an NDN-native Cache Pollution Attack generator within the Docker testbed, with adversarial consumers issuing Interest patterns matching the Locality-Disruption and False-Locality templates introduced in Section 1.2.2. Capturing the resulting traffic and adding cache pollution as a sixth attack class would close the gap between the threat-landscape framing of Chapter 1 and the empirical evaluation of the present work, and would directly exercise the topological-coordination reasoning capacity of the third-tier graph neural network on the precise category of collusive coordination that motivated its architectural design. A containerised implementation would fit naturally within the existing Attack Generation Framework, with synthetic adversarial consumers orchestrated through the same Docker bridge network used by the current five-class generator.

DETAILED SOFTWARE MODULE DESCRIPTIONS

The architectural and methodological commitments documented across the body of the thesis are realised in software modules whose specific roles within the overall system pipeline deserve explicit cataloguing for the benefit of readers seeking to reproduce, extend, or audit the implementation. The present appendix maps each component documented in the main text to the corresponding module in the source repository, with the role of each module within the architectural design briefly characterised.

The repository organisation places the per-tier training and inference modules under their respective tier-named subdirectories, the data engineering pipeline under the project root with the dataset-curation modules consuming the outputs of the feature-extraction modules in pipeline order, and the orchestration and benchmarking modules at the project root for direct invocation against the trained model artefacts. The configuration parameters governing each module's operation, including the seventeen-feature schema, the model hyperparameters, and the escalation thresholds, are exposed at the module level rather than hardcoded into the implementation, supporting both reproducibility of the reported results and extension of the architecture under modified parameter configurations.

Table 1: Software modules and their architectural roles.

Module	Role	Reference
`attack_lab/`	Containerised testbed orchestration directory containing the Docker Compose specifications, producer container definitions, adversarial container definitions, and the bridge network configuration.	Sec. 3.1.2, Sec. 3.3
`capture_large.sh`	Packet capture orchestration script that initiates and terminates per-session `tcpdump` instances on producer container virtual interfaces, with session boundaries aligned to attack-script invocation for deterministic labelling.	Sec. 3.2.1
`capture.sh`	Per-container capture utility executed within producer containers to invoke `tcpdump` with the appropriate Berkeley Packet Filter expression and output-path specification.	Sec. 3.2.1
`generate_synthetic_attacks.py`	Statistical-envelope-based attack traffic generator implementing the five-class production taxonomy with the per-class timing and payload distributions documented in the synthetic attack generation methodology.	Sec. 3.3.1, Sec. 3.3.2
`extract_v5_features.py`	Per-packet feature extraction module that computes the seventeen-dimensional feature vector from raw PCAP frame contents, including the Shannon entropy computation on payload bytes.	Sec. 3.2.2, Sec. 3.2.3
`extract_flow_features.py`	Flow-level aggregate feature computation module implementing the group-and-transform operation over the destination-port-and-protocol tuple to produce the flow-aggregate components of the feature vector.	Sec. 3.2.2
`build_v5_final.py`	Multi-stage dataset curation pipeline that consumes the extracted features, applies the leakage-prevention filtering, partitions the data under the `GroupShuffleSplit` protocol, and produces the model-ready training, validation, and test partitions.	Sec. 3.4.1, Sec. 3.4.2
`data_detox.py`	Pre-curation cleaning module that filters non-IP frames, eliminates zero-variance columns, and removes the strongly class-correlated features identified during the leakage diagnosis.	Sec. 3.4.1
`refine_dataset.py`	Sequence construction module that converts the per-packet feature stream into the temporally ordered 20×17 window representation under the within-flow-group windowing constraint with stride ten.	Sec. 3.4.2

Table 2: Software modules and their architectural roles (continued).

Module	Role	Reference
`tier1/train_rf_v4.py`	First-tier Random Forest training module implementing the $T = 200$ ensemble with class-weight reweighting and unrestricted per-tree depth.	Sec. 4.1.1
`tier2/train_cnn_gru_v4.py`	Second-tier convolutional-recurrent classifier training module implementing the Conv1D $\rightarrow$ GRU $\rightarrow$ FC architecture under Focal Loss with the production hyperparameter configuration.	Sec. 4.1.2, Sec. 4.2, Sec. 4.3
`tier2/train_gnn_v1.py`	Third-tier graph neural network training module implementing the dynamic aggregation-operator selection between graph attention and graph sample-and-aggregate at the five-hundred-vertex threshold.	Sec. 4.1.3, Sec. 4.4
`decision/router.py`	Cascaded escalation logic implementation that orchestrates the per-tier inference dispatch, applies the confidence-threshold escalation conditions, and produces the final pipeline output with source-tier annotation.	Sec. 4.1.4
`benchmark_latency.py`	Production latency benchmarking module that measures per-tier and aggregate inference cost across the test partition, generates the comparative measurement against the Mamba state-space-model baseline, and produces the latency-distribution outputs analysed in the results chapter.	Sec. 5.5
`run_benchmark.py`	Orchestration script for the end-to-end performance evaluation that invokes the latency benchmark, computes the per-class precision and recall metrics, and produces the proof-of-work logs that constitute the empirical record of the present work.	Sec. 5.3

REFERENCES

1. Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.

2. Kyunghyun Cho, Bart van Merriënboer, Çaglar Gülçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *Proceedings of EMNLP*, 2014.

3. Alberto Compagno, Mauro Conti, Paolo Gasti, and Gene Tsudik. Poseidon: Mitigating interest flooding DDoS attacks in named data networking. In *38th IEEE Conference on Local Computer Networks (LCN)*, pages 630–638, 2013.

4. Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *Conference on Language Modeling (COLM)*, 2024.

5. William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

6. Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017.

7. Kathleen Nichols and Van Jacobson. Controlling queue delay. *Communications of the ACM*, 55(7):42–50, 2012.

8. Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

9. Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

10. Lixia Zhang, Alexander Afanasyev, Jeffrey Burke, Van Jacobson, KC Claffy, Patrick Crowley, Christos Papadopoulos, Lan Wang, and Beichuan Zhang. Named data networking. *ACM SIGCOMM Computer Communication Review*, 44(3):66–73, 2014.